

2



Architectural Design

We now dive into our discussion of architecture design, a complex and much misunderstood topic. We will spend time talking about what it is, why it is important, how it works (at an abstract level), and which major concepts and activities it involves. We will also discuss architectural drivers: the various factors that “drive” design decisions, some of which may be documented as requirements, but many of which will not be.

Why Read This Chapter?

Design is the core concept of this book. Designs can be arbitrarily complex but you can’t possibly deal with all of the complexity all of the time. This chapter gives you a way to think about design decisions at different levels of abstraction, which is key to taming the complexity. Also, you may not intuitively consider all dimensions of drivers. Here we give you a framework for thinking about drivers; we will continue to use this framework throughout the rest of the book.

2.1 Design in General

Design is both a verb and a noun. Design is a process, an activity, and hence a verb. This process results in the creation of a design—a description of a desired state. The output of the design process is the thing, the noun, the artifact that you will eventually implement. Designing means making decisions to achieve goals and to satisfy requirements and constraints. The outputs of the design process are a direct reflection of those goals, requirements, and constraints. Think about houses, for example. Why do traditional houses in China look different from those in Switzerland or Algeria? Why does a yurt look like a yurt, which is different from an igloo or a chalet or a longhouse?

The architectures of these styles of houses have evolved over the centuries to reflect their unique sets of goals, requirements, and constraints. Houses in China feature symmetric enclosures, sky wells to increase ventilation, south-facing courtyards to collect sunlight and provide protection from cold north winds, and so forth. A-frame houses have steep pitched roofs that

extend to the ground, meaning minimal painting and protection from heavy snow loads (which just slide off to the ground). Igloos are built of ice, reflecting the availability of ice, the relative poverty of other building materials, and the constraints of time (a small one can be built in an hour).

In each case, the process of design involves the selection and adaptation of a number of solution approaches. For example, igloo designs can vary. Some are small and meant to serve as a temporary travel shelter. Others are large, often connecting several structures, meant for entire communities to meet. Some are simple unadorned snow huts. Others are lined with furs, with ice “windows” and doors made of animal skin.

The process of design, in each case, balances the various “forces” facing the designer. Some designs require considerable skill to execute (such as carving and stacking snow blocks in such a way that they produce a self-supporting dome). Others require relatively little skill—a simple lean-to can be constructed from branches and bark by almost anyone. But the qualities that these structures exhibit may also vary considerably. Lean-tos provide little protection from the elements and are easily destroyed by high winds or fire, whereas an igloo can withstand Arctic storms and support the weight of a person standing on the roof.

Is design “hard”? Well, yes and no. *Novel* design is hard. It is pretty clear how to design a conventional bicycle, but the designs for the Segway and the OneWheel broke new ground. Fortunately, most design is not novel, because most of the time our requirements are not novel. Most people want a bicycle that will reliably convey them from place to place. Most people living in Phoenix want a house that can be easily and economically kept cool, whereas most people in Edmonton are primarily concerned with a house that can be kept warm. In contrast, people living in Tokyo and Mexico City are often concerned with buildings that can withstand serious earthquakes.

The good news for you, the architect, is that there are ample proven designs and design fragments, or building blocks that we call *design concepts*, that can be reused and combined to reliably achieve these goals. If your design is truly novel—if you are designing the next Sydney Opera House—then the design process will likely be “hard”. The Sydney Opera House, for example, cost nearly 14 times its original budget estimate and was delivered 10 years late. So, too, with the design of software architectures.

2.2 Design in Software Architecture

Architectural design for software systems is no different than design in general: It involves making decisions, working with available skills and materials, to satisfy requirements and constraints. In architectural design, we make decisions to transform our design purpose, primary functional requirements, quality attribute requirements, constraints, and architectural concerns—what we call the architectural *drivers*—into structures, as shown in Figure 2.1. These structures, which are mentioned in the definition of software architecture that we gave in Chapter 1, are then used to guide the project. They guide analysis and construction, and serve as the foundation for educating a new project member. They also guide cost and schedule estimation, team formation, risk analysis and mitigation, and, of course, implementation.

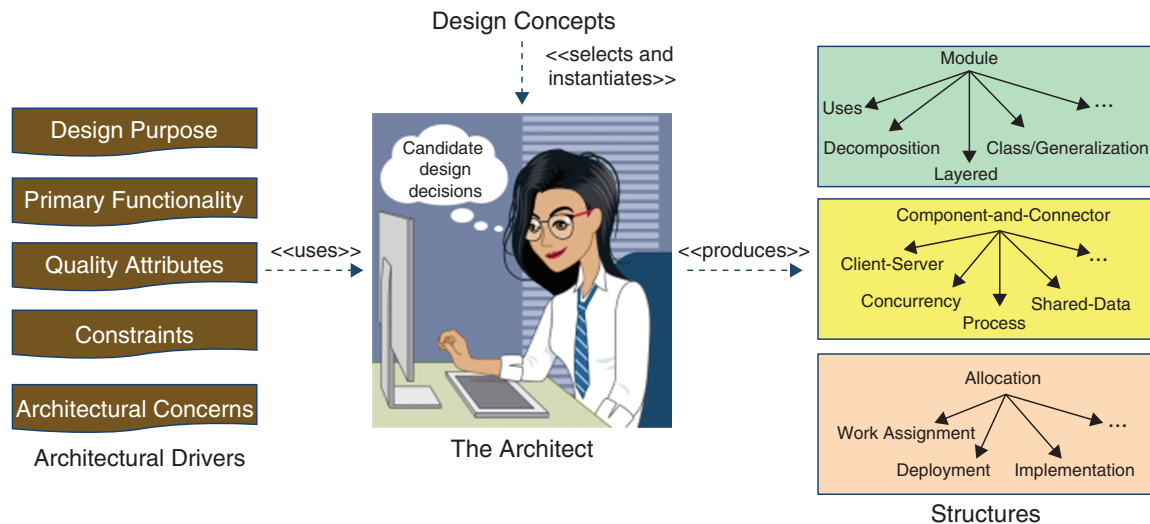


FIGURE 2.1 Overview of the architecture design activity

Architectural design is, therefore, a key step to achieving your product and project goals. Some of these goals are technical (e.g., achieving low and predictable latency in a video game or an e-commerce website), and some are nontechnical (e.g., supporting multiple lines of business, keeping your existing team employed, entering a new market, meeting a deadline). The decisions that you, as an architect, make will have implications for the achievement of these goals and may, in some cases, be in conflict. The choice of a particular reference architecture (e.g., the Lambda architecture) may provide a good foundation for achieving your latency and throughput goals and will keep your workforce employed because they are already familiar with that reference architecture and its supporting technology stacks. But this choice will not help you enter a new market—mobile games, for example.

In general, during the design process, a change in some structure to achieve one quality attribute will have negative effects on other quality attributes. These *tradeoffs* are a fact of life for every practicing architect in every domain. We will see this over and over again in the examples and case studies provided in this book. In essence, the architect's job is not one of finding an optimal solution, but rather one of *satisficing*—searching through a potentially large space of design alternatives and decisions until an acceptable solution is found.

2.2.1 Architectural Design

Grady Booch has said, “All architecture is design, but not all design is architecture”. What makes a decision “architectural”? A decision is architectural if it has non-local consequences *and* those consequences matter to the achievement of an architectural driver. No decision is, therefore, inherently architectural or non-architectural. The choice of a buffering strategy

within a single element may have little effect on the rest of the system, in which case it is an implementation detail that is of no concern to anyone except the implementer or maintainer of that element. In contrast, the buffering strategy may have enormous implications for overall system performance (if the buffering affects the achievement of latency or throughput or jitter goals) or availability (if the buffers might not be large enough and information gets lost) or modifiability (if we wish to flexibly change the buffering strategy in different deployments or contexts). The choice of a buffering strategy, like most design choices, is neither inherently architectural nor inherently non-architectural. Instead, this distinction is completely dependent on the current and anticipated architectural drivers.

2.2.2 Element Interaction Design

Architectural design identifies and defines the major elements and important relationships that make up the overall structure of a system. The major elements are often abstract or existing design concepts (such as architectural patterns) that address primary functional requirements, quality attribute requirements, constraints, and architectural concerns. Architectural design generally results in the identification of only a subset of the elements that are part of the system's structure. This is to be expected because, during initial architectural design, the architect will focus on the primary functionality of the system.

What makes a use case primary? A combination of business importance, risk, and complexity considerations feed into this designation. Of course, to the typical user almost everything will be urgent and the number one priority. More realistically, a small number of user stories provide the most fundamental business value or represent the greatest risk (if they are done wrong), so these are deemed primary. Every system has many more user stories, beyond the primary ones, that need to be satisfied. The elements that support these nonprimary user stories and their interfaces are part of what we call *element interaction design*. This level of design usually follows the overall architectural design. The precise locations and relationships of these elements, however, are constrained by the decisions that were made during architectural design. These elements can be units of work (i.e., modules or services) assigned to an individual or to a team, so this level of design is important not only for defining how non-primary functionality is allocated, but also for planning purposes (e.g., team formation and communication, budgeting, outsourcing, release planning, unit and integration test planning).

Depending on the scale and complexity of the system, the architect should be involved in element interaction design, either directly or in an auditing role. This involvement ensures that the system's important quality attributes are not compromised—for example, that the elements are not defined, located, or connected incorrectly. It also helps the architect spot opportunities for generalization.

2.2.3 Element Internals Design

A third level of design follows element interaction design, which we call *element internals design*. In this level of design, which is usually addressed as part of the element development

activities, the internals of the elements identified in the previous design level are established so as to satisfy the element's interface.

2.2.4 Decisions and Design Levels

Architectural decisions can and do occur at the three levels of design. Moreover, during architectural design, the architect may need to delve as deeply as element internals design to achieve a particular architectural driver. An example is the design of the internals of a microservice following a layered architecture. The data layer of the microservice may need to be carefully designed so that it does not contribute excessive latency or lock important parts of its database. In this sense, architectural design can, at times, involve considerable detail, which explains why we do not like to talk about it in terms of “high-level design” or “detailed design” (see the sidebar “High-Level vs. Detailed Design?”).

Architectural design precedes element interaction design, which precedes element internals design. This is logically necessary: You cannot design an element's internals until you have defined the elements themselves, and you cannot usefully reason about interactions until you have defined several elements and some patterns of interactions among them. But as projects or products grow and evolve, there is, in practice, considerable iteration between these activities.

High-Level vs. Detailed Design?

The term “detailed design” is often used to refer to the design of the internals of modules. Although it is widely used, we really don't like this term, which is presented as somehow in opposition to “high-level design”. We prefer the more precise terms “architectural design”, “element interaction design”, and “element internals design”.

After all, architectural design may be quite detailed, if your system is complex. And some design “details” will turn out to be architectural. For the same reason, we also don't like the terms “high-level design” and “low-level design”. Who can really know what these terms mean? Clearly, “high-level design” should be somehow “higher” or more abstract, and cover more of the architectural landscape than “low-level design”, but beyond that we are at a loss to imbue these terms with any precise meaning.

So here is what we recommend: Just avoid using terms such as “high”, “low”, or “detailed” altogether. There is always a better, more precise choice, such as “architectural”, “element interaction”, or “element internals” design!

Think carefully about the impact of the decisions you are making, the information that you are trying to convey in your design documentation, and the likely audience for that information, and then give that process an appropriate, meaningful name.

2.3 Why Is Architectural Design So Important?

Although premature commitment can be wasteful, and sometimes “you ain’t gonna need it”, there is a very high cost to a project of *not* making important design decisions, or of not making them early enough. This manifests itself in many different ways. Early on, an initial architecture is critical for estimation in project proposals. Without doing some architectural thinking and some early design work, you cannot confidently predict project cost, schedule, and quality. Even at this early stage, an initial architecture will determine the key approaches for achieving architectural drivers, the gross work-breakdown structure, and the choices of tools, skills, and technologies needed to realize the system.

In addition, architecture is a key enabler of agility, as we will discuss in Chapter 12. Whether your organization has embraced Agile processes or not, it is difficult to imagine anyone who would willingly choose an architecture that is brittle and hard to change or extend or tune—and yet it happens all the time. This kind of *technical debt* (which we discuss in Chapter 10) occurs for a variety of reasons, but paramount among these is the combination of a focus on features—typically driven by stakeholder demands—and the inability of architects and project managers to measure the return on investment of good architectural practices. Features provide immediate benefit. Architectural improvement provides immediate costs and, if done right, long-term benefits. Put this way, why would anyone ever “invest” in architecture? The answer is simple: Without architecture, the benefits that the system is supposed to bring will be far harder to realize; velocity will slow and the system will become increasingly more difficult to debug, fix, scale, and evolve.

Simply put, if you do not make some key architectural decisions early on, and if you allow your architecture to degrade, you will be unable to maintain sprint velocity, because you cannot easily respond to change requests. We vehemently disagree with what the original creators of the Agile Manifesto claimed: “The best architectures, requirements, and designs emerge from self-organizing teams”. Indeed, our demurral with this point is precisely why we have written this book. Good architectural design is difficult (and still rare), and it does not just “emerge”. This opinion mirrors a growing consensus within the Agile community. More and more, we see techniques such as “disciplined agility at scale”, the “walking skeleton”, and the “Scaled Agile Framework” embraced by Agile thought leaders and practitioners alike. Each of these techniques advocates some architectural thinking and design prior to much, if any, development. To reiterate, architecture enables agility, and not the other way around.

Furthermore, the architecture will influence, but not determine, other decisions that are not in and of themselves design decisions. These decisions do not influence the achievement of quality attributes directly, but still need to be made by the architect. For example, such decisions may include selection of tools, structuring the development environment, and making work assignments.

Finally, a well-designed, properly communicated architecture is key to achieving *agreements* that will guide the team. The most important kinds of consensus to reach are agreements on interfaces and on shared resources. Agreeing on interfaces early is important for component-based development, and critically important for distributed development using approaches such as microservices. These decisions *will* be made sooner or later. If you don’t

make them early in the design process, the system will be much more difficult to integrate. In Section 4.6, we will discuss how to define interfaces as part of architectural design—both the external interfaces to other systems and the internal interfaces that mediate your element interactions.

2.4 Architectural Drivers

Before commencing design with ADD (or with any other design approach, for that matter), you need to think about what you are doing and why. Although this statement may seem blindingly obvious, the devil is, as usual, in the details. We categorize these “what” and “why” questions as architectural drivers. As shown in Figure 2.1, these drivers include a design purpose, quality attributes, primary functional requirements, architectural concerns, and constraints. These considerations are critical to the success of the system and, as such, they *drive* and shape the architecture.

As with any other important requirements, architectural drivers need to be baselined and managed throughout the development life cycle.

2.4.1 Design Purpose

First, you need to be clear about the purpose of the design that you want to achieve. When and why are you doing this architecture design? Which business goals is the organization most concerned about at this time?

1. You may be doing architecture design as part of a project proposal (for the estimation process in a consulting organization, or for internal project selection and prioritization in a company, as discussed in Section 12.1.1). It is not uncommon that, as part of determining project feasibility, schedule, and budget, an initial architecture is created. Such an architecture would not be very detailed; the goal is to understand and break down the architecture in sufficient detail that the units of work are understood and hence may be estimated.
2. You may be doing architecture design as part of the process of creating an exploratory prototype. In this case, the purpose of the architecture design process is not so much to create a releasable or reusable system, but rather to explore the domain, to explore new technology, to place something executable in front of a customer to elicit rapid feedback, or to explore some quality attribute (such as latency or performance scalability or failover for availability).
3. You may be designing your architecture during development. This could be for a complete new system, for a substantial portion of a new system, or for a portion of an existing system that is being refactored or replaced. In this case, the purpose is to do just enough design work to satisfy requirements, guide system construction and work assignments, and prepare for an eventual release.

In a mature domain, the estimation process might be relatively straightforward; the architect can reuse existing systems as examples and confidently make estimates based on analogy. In novel domains, the estimation process will be far more complex and risky, and may have highly variable results. In these circumstances, a prototype of the system, or a key part of the system, may need to be created to mitigate risk and reduce uncertainty. In many cases, this architecture may also need to be quickly adapted as new requirements are learned and embraced. In brown-field systems, where the requirements are better understood, the existing system is itself a complex artifact that must be well understood for planning to be accurate.

The development organization's goals during development or maintenance may also affect the architecture design process. For example, the organization might be interested in designing for reuse, designing for future extension or subsetting, designing for scalability, designing a product for continuous delivery, designing to best utilize existing project capabilities and team member skills, and so forth. Or the organization might have a strategic relationship with a vendor. Or the CTO might have a specific technology preference and wants to impose it on your project.

Why do we bother to list these considerations? Because they *will* affect both the process of design and the outputs of design. Architectures exist to help achieve business goals. The architect should be clear about these goals and should communicate them (and negotiate them!) and establish a clear design purpose *before* beginning the design process.

2.4.2 Quality Attributes

In the book *Software Architecture in Practice*, *quality attributes* are defined as being measurable or testable properties of a system that are used to indicate how well the system satisfies the needs of its stakeholders. Because the term *quality*, as it is informally used, tends to be a squishy term, these properties allow quality to be expressed succinctly and objectively.

Among the drivers, quality attributes are the ones that most strongly shape the architecture. The critical choices that you make when you are doing architectural design determine, in large part, the ways that your system will or will not meet these driving quality attribute goals.

Given their importance, you must worry about eliciting, specifying, prioritizing, and validating quality attributes. Given that so much depends on getting these drivers right, this sounds like a daunting task. Fortunately, a number of well-understood, widely disseminated practices can help you here (see the sidebar “The Quality Attribute Workshop and the Utility Tree” for a more detailed discussion of the first two techniques):

- A *Utility Tree*, the simplest technique, can be used by the architect to quickly prioritize quality attribute requirements according to their technical difficulty and risk.
- The *Quality Attribute Workshop (QAW)*, a bit more costly and complex, is a facilitated brainstorming session involving a group of system stakeholders that covers the bulk of the activities geared toward eliciting, specifying, prioritizing, and achieving consensus on quality attributes.
- The *Mission Thread Workshop* serves the same purpose as QAW, but for a system of systems. It is typically the most costly, most complex method. However, if you are building a system of systems, the greater effort and expense are probably worth it.

The best way to discuss, document, and prioritize quality attribute requirements is as a set of scenarios. A *scenario*, in its most basic form, describes the system’s response to some stimulus. Why are scenarios the best approach? Because all other approaches are worse! Endless time may be wasted in defining terms such as “performance” or “scalability” or “modifiability” or “configurability”, even though these discussions tend to shed little light on the real system. It is meaningless to say that a system will be “modifiable”, because every system is modifiable with respect to some changes and not modifiable with respect to others. You can, however, specify the modifiability response measure you would like to achieve as the result of a specific change request, perhaps in terms of elapsed time or total effort. For example, you might want to specify that “a request to update shipping rates is processed in less than 50 milliseconds”—an unambiguous criterion.

The heart of a quality attribute scenario, therefore, is the pairing of a stimulus with a response. Suppose that you are building a video game and you have a functional requirement like this: “The game shall change view modes when the user presses the <C> button”. This functional requirement, if it is important, needs to be associated with quality attribute requirements. For example:

- How fast should the function be?
- How secure should the function be?
- How modifiable should the function be?

To address this problem, we use a *scenario* to describe a quality attribute requirement. A quality attribute scenario is a short description of how a system is required to respond to some stimulus. For example, we might annotate the functional requirement given earlier as follows: “The game shall change view modes in less than 500 ms when the user presses the <C> button”. A scenario associates a stimulus (in this case, pressing the <C> button) with a response (changing the view mode) that is measured using a response measure (less than 500 ms). A complete quality attribute scenario adds three other parts: the source of the stimulus (in this case, the user), the artifact affected (in this case, because we are dealing with end-to-end latency, the artifact is the entire system), and the environment (are we in normal operation, startup, degraded mode, or some other mode?). In total, then, there are six parts of a completely well-specified scenario, as shown in Figure 2.2.

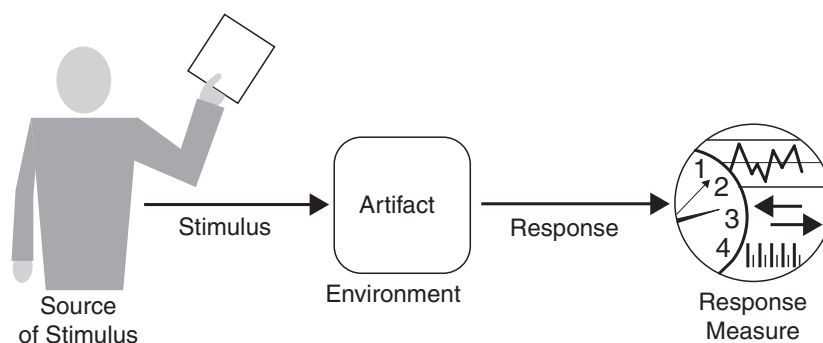


FIGURE 2.2 The six parts of a quality attribute scenario

Scenarios are testable, *falsifiable hypotheses* about the quality attribute behavior of the system under consideration. Because they have explicit stimuli and responses, we can evaluate a design in terms of how likely it is to support the scenario, and we can take measurements and test a prototype or fully fleshed-out system for whether it satisfies the scenario in practice. If the analysis (or prototyping results) indicates that the scenario's response goal likely won't be met, then the hypothesis is not supported.

We often hear the objections "I don't know precisely what that requirement is" and "I don't know what that response measure should be". You should still collect and document scenarios, even in the face of uncertainty, for two reasons: (1) just struggling with this question, and perhaps discussing it, will help to clarify the requirement, and (2) even if the precise requirements are not known, you probably know the value of the response measure to within an order of magnitude. If the system should respond "quickly" to a user's request for a report, you may not know whether 2 seconds or 5 seconds or 10 seconds is the best number to use as the cutoff, but you certainly know that 100 seconds doesn't count as "quickly" in most people's books. This kind of order-of-magnitude requirement is typically enough to guide architectural decisions.

As with all other requirements, scenarios should be prioritized. This can be achieved by considering two dimensions that are associated with each scenario:

- The first dimension corresponds to the importance of the scenario with respect to the success of the system. This is ranked by the customer.
- The second dimension corresponds to the degree of technical risk associated with the scenario. This is ranked by the architect.

A low/medium/high (L/M/H) scale is used to rank both dimensions; in practice, most people don't have too much trouble making these coarse distinctions. Once the dimensions have been ranked, scenarios are prioritized by selecting those that have a (H, H) ranking. If there aren't enough of those, then we look at scenarios with (H, M), or (M, H) rankings. Why these ones? If a scenario involves high business importance *and* high risk, it is definitely a driver! And it certainly deserves your attention.

In addition, some traditional requirements elicitation techniques can be modified slightly to focus on quality attribute requirements. Those techniques include Joint Requirements Planning (JRP), Joint Application Design (JAD), discovery prototyping, and user stories.

No matter which technique you use, *do not* start design without a prioritized list of measurable quality attributes! Although stakeholders might plead ignorance ("I don't know how fast it needs to be; just make it fast!"), you can always elicit at least a range of possible responses. Instead of accepting the requirement that the system should be "fast", ask the stakeholder if a 10-second response time is acceptable. If that is unacceptable, ask if 5 seconds is OK, or, failing that, 1 second. You will find that, in most cases, users know more than they realize about their requirements, and you can at least "box them in" to a range.

The Quality Attribute Workshop and the Utility Tree

The Quality Attribute Workshop (QAW)

The QAW is a facilitated, stakeholder-focused method that can be used to generate, prioritize, and refine quality attribute scenarios. A QAW meeting is ideally enacted before the software architecture has been defined, although, in practice, we have seen the QAW being used at all points in the software development life cycle. The QAW focuses on system-level concerns and specifically the role that software will play in the system. The steps of the QAW are as follows:

1. QAW Presentation and Introductions

The QAW facilitators describe the motivation for the QAW and explain each step of the method.

2. Business Goals Presentation

A stakeholder representing the project's business concerns presents the system's business context, broad functional requirements, constraints, and known quality attribute requirements. The quality attributes that will be refined in later QAW steps will be derived from, and should be traceable to, the business goals presented in this step. For this reason, these business goals must be prioritized.

3. Architectural Plan Presentation

The architect presents the system architectural plans as they currently exist. Although the architecture may not have been fully defined yet (particularly for green-field systems), the architect often knows quite a lot about it even at this early stage. For example, the architect might already know about technologies that are mandated, other systems that this system must interact with, standards that must be followed, subsystems or components that could be reused, and so forth.

4. Identification of Architectural Drivers

The facilitators share the list of key architectural drivers that they assembled during steps 2 and 3 and ask the stakeholders for clarifications, additions, deletions, and corrections. The idea here is to reach a consensus on a distilled list of architectural drivers that covers major functional requirements, business drivers, constraints, and quality attributes.

5. Scenario Brainstorming

Given this context, each stakeholder now has the opportunity to express a scenario representing their needs and desires with respect to the system. The facilitators ensure that each scenario has an explicit stimulus and response. The facilitators also ensure traceability and completeness: At least one representative scenario should exist for each architectural driver listed in step 4, and collectively the scenarios should cover all of the business goals listed in step 2.

6. Scenario Consolidation

Similar scenarios are consolidated where reasonable. In step 7, the stakeholders vote for their favorite scenarios, and consolidation helps to prevent votes from being spread across several scenarios that are expressing essentially the same concern.

7. Scenario Prioritization

Prioritization of the scenarios is accomplished by allocating to each stakeholder a number of votes equal to 30 percent of the total number of scenarios. The stakeholders can distribute these votes to any scenario or scenarios. Once all the stakeholders have voted, the results are tallied and the scenarios are sorted in order of popularity.

8. Scenario Refinement

The highest-priority scenarios are refined and elaborated. The facilitators help the stakeholders express these in the form of six-part scenarios: source, stimulus, artifact, environment, response, and response measure.

In summary, the output of the QAW is a prioritized list of scenarios, aligned with business goals, where the highest-priority scenarios have been explored and refined. A QAW can be conducted in as little as 2–3 hours for a simple system or as part of an iteration, but may last as long as 2 days for a complex system where requirements completeness is a goal.

The Utility Tree

If no stakeholders are readily available to consult, you still need to decide what to do and how to prioritize the many challenges facing the system. One way to organize your thoughts is to create a Utility Tree. A Utility Tree, such as the one shown in Figure 2.3, helps to articulate your quality attribute goals in detail, and then to prioritize them.

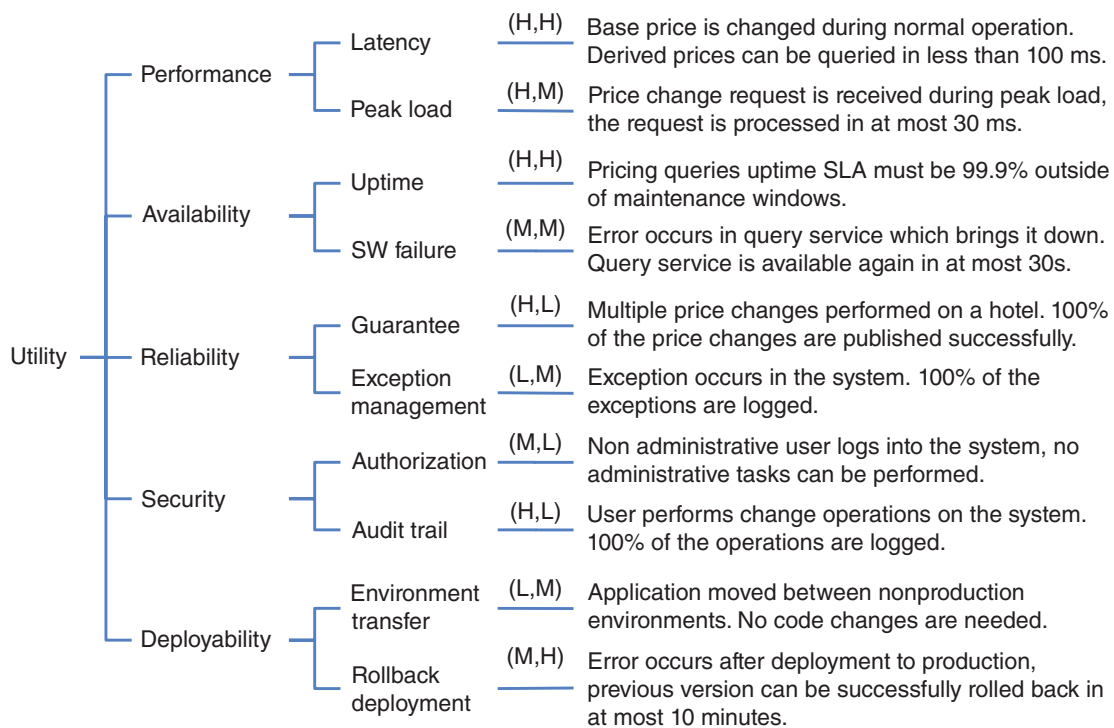


FIGURE 2.3 A Utility Tree

It works as follows. First write the word “Utility” on a sheet of paper. Then write the various quality attributes that constitute utility for your system. For example, you might know, based on the business goals for the system, that the most important qualities are that the system is fast, secure, and easy to modify. In turn, you would write these words underneath “Utility”. Next, because you don’t really know what any of those terms actually means, you describe the aspect of the quality attribute that you are most concerned with. For example, “performance” is vague, but “latency of price changes” is a bit less vague. Likewise, “deployability” is vague, but “rollback deployment” is a bit less vague.

The leaves of the tree are expressed as scenarios, which provide concrete examples of the quality attribute considerations that you just enumerated. For example, for “latency of price changes”, you might create a scenario such as “Base price is changed during normal operation. Derived prices can be queried in less than 100 ms”. For “deployability”, you might create a scenario such as “Error occurs after deployment to production, previous version can be successfully rolled back in at most 10 minutes”.

Finally, prioritize the scenarios that you have created. We perform this prioritization by using the technique of ranking across two dimensions, resulting in a priority matrix such as the following (where the numbers in the cells are from a set of system scenarios).

BUSINESS IMPORTANCE/ TECHNICAL RISK	L	M	H
L	5, 6, 17, 20, 22	1, 14	12, 19
M	9, 12, 16	8, 20	3, 13, 15
H	10, 18, 21	4, 7	2, 11

Our job, as architects, is to focus on the lower-right-hand portion of this table (H, H): those scenarios that have high business importance and high risk. Once we have satisfactorily addressed those scenarios, we can move to the (M, H) or (H, M) ones, and then move up and to the left until all of the system’s scenarios are addressed (or perhaps until we run out of time or budget, as is often the case).

The QAW and the Utility Tree are different techniques that share the same goal—eliciting and prioritizing the most important quality attribute requirements, which will be some of your most critical architectural drivers. There is no reason, however, to choose between these techniques—both are useful and valuable. In our experience, they have complementary strengths: The QAW tends to focus more on the requirements of external stakeholders, whereas the Utility Tree tends to excel at eliciting the requirements of internal stakeholders. Making all of these stakeholders happy will go a long way toward ensuring the success of your architecture.

2.4.3 Primary Functionality

Functionality is the ability of the system to do the work for which it was intended. As opposed to quality attributes, the way the system is structured does not normally influence functionality. You can have all of the functionality of a given system coded in a single enormous module (a god class, for example), or you can have it neatly distributed across many smaller, highly cohesive modules. Externally the system will look and work the same way if you consider only functionality. What matters, though, is what happens when you want to make changes to such a system. In the former case, changes will be difficult and costly; in the latter case, they

should be much easier and cheaper to perform. In terms of architectural design, allocation of functionality to elements, rather than the functionality per se, is what matters. A good architecture is one in which the most common changes are localized in a single or a few elements (as a consequence of high cohesion), and hence easy to make.

When designing an architecture, you need to consider at least the primary functionality. Primary functionality is usually defined as functionality that is critical to achieve the business goals that motivate the development of the system. These are typically captured as user stories or use cases (we will use these terms interchangeably throughout this book). Primary functionality is often focused on the “happy” paths, where everything works as planned and the system delivers some useful outcome. Other criteria for primary functional requirements might be that it implies a high level of technical difficulty or requires the interaction of many architectural elements. As a rule of thumb, approximately 10 percent of your user stories or use cases are likely to be primary.

There are two important reasons why you need to consider primary functionality when designing an architecture:

1. You need to think how functionality will be allocated to elements (usually modules) to promote modifiability or reusability, and also to plan work assignments.
2. Some quality attribute scenarios are directly connected to the primary functionality in the system. For example, in a movie streaming application, one of the primary use cases is, of course, to watch a movie. This use case is associated with a performance quality attribute scenario such as “Once the user clicks ‘Play’, the movie should begin streaming in no more than 5 seconds”. In this case, the quality attribute scenario is directly associated with the primary user story, so making decisions to support this scenario also requires making decisions about how its associated functionality will be supported. This is not the case for all quality attributes, however. For example, an availability scenario can involve recovery from a system failure, and this failure may occur when any of the system’s functions is being executed.

Decisions regarding the allocation of functionality that are made during architectural design establish a precedent for how the rest of the functionality should be allocated to modules as development progresses. This is usually not the work of the architect; instead, this activity is typically performed as part of the element interaction design process described in Section 2.2.2.

Finally, bad decisions about the allocation of functionality result in the accumulation of technical debt. This debt can be paid through the use of refactoring, although this impacts the project’s rate of progress, or velocity (see Chapter 10).

2.4.4 Architectural Concerns

Architectural concerns encompass additional aspects that need to be considered as part of architectural design but are not expressed as traditional requirements. There are several different types of concerns:

- *General concerns.* These are “broad” issues that you deal with in creating the architecture, such as establishing the overall system structure, the allocation of functionality to modules, the allocation of modules to teams, and the organization of the code base.

- *Specific concerns.* These are more detailed, system-internal issues such as exception management, dependency management, configuration, logging, authentication, authorization, caching, and so forth that are common across large numbers of applications. Some specific concerns are addressed in reference architectures (see Section 3.2.1), but others will be unique to your system. Specific concerns may also result from previous design decisions. For example, you may need to address API versioning if you previously decided to use a REST API.
- *Internal requirements.* These requirements are usually not specified explicitly in traditional requirement documents, as customers usually seldom express them. Internal requirements may address aspects that facilitate development, deployment, operation, or maintenance of the system. They are sometimes called “derived requirements”. An example would be the use of a particular data type to manage currencies, or the requirement to store all timestamps in UTC.
- *Issues.* These result from analysis activities, such as a design review (see Section 11.6), so they may not be present initially. For instance, an architectural evaluation may uncover a security risk that requires some changes to the current design.

Some of the decisions surrounding architectural concerns might be trivial or obvious. For example, your deployment structure might be a single processor for an embedded system, or a single cell phone for an app. Your reference architecture might be constrained by company policy. Your authentication and authorization policies might be dictated by your enterprise architecture and realized in a shared framework. In other cases, however, the decisions required to satisfy particular concerns may be less obvious—for example, decisions involving exception management or input validation or structuring the code base.

From their past experience, wise architects are usually aware of the concerns that are associated with a particular type of system and the need to make early design decisions to address them. Inexperienced architects are usually less aware of such concerns; because these concerns tend to be tacit rather than explicit, they may not consider them as part of the design process, which often results in problems later on.

Architectural concerns frequently result in the introduction of new quality attribute scenarios. The general concern of “supporting logging”, for example, is too vague and needs to be made more specific. Like the quality attribute scenarios that are provided by the customer, these scenarios need to be prioritized. For these scenarios, however, the customer is the development team, operations, or other members of the organization. During design, the architect must consider both the quality attribute scenarios that are provided by the customer and the scenarios that are derived from architectural concerns.

One of our goals when revising the ADD method was to elevate the importance of architectural concerns as explicit inputs to the architecture design process. That issue will be highlighted in our examples and case studies in Chapters 8 and 9.

2.4.5 Constraints

As part of the architectural design process, you need to catalog the constraints on development. These constraints may take the form of mandated technologies, other systems with which your system needs to interoperate or integrate, laws and standards that the system must comply with,

the abilities and availability of your developers, deadlines that are non-negotiable, backward compatibility with older versions of systems, and so on. An example of a technical constraint is the use of open source technologies, whereas a nontechnical constraint is that the system must obey the Sarbanes-Oxley Act or that it must be delivered by December 15.

A constraint is a decision over which you have little or no control as an architect. Your job is, as we mentioned in Chapter 1, to *satisfice*: to design the best system that you can, despite the constraints you face. Sometimes you might be able to argue for loosening a constraint, but in most cases you have no choice but to design around the constraints.

2.5 Summary

In this chapter, we introduced the idea of design as a set of decisions to satisfy requirements and constraints. We also introduced the notion of “architectural” design and showed that it does not differ from design in general, other than that it addresses the satisfaction of *architectural drivers*: the purpose, primary functional requirements, quality attribute requirements, architectural concerns, and constraints. What makes a decision “architectural”? A decision is architectural if it has non-local consequences *and* those consequences matter to the achievement of an architectural driver.

We also discussed why architectural design is so important: because it is the embodiment of early, far-reaching, hard-to-change decisions. These decisions will help you meet your architectural drivers, will determine much of your project’s work-breakdown structure, and will affect the tools, skills, and technologies needed to realize the system. Thus, architectural design decisions should be scrutinized closely, as their consequences are profound. In addition, architecture is a key enabler of agility.

2.6 Further Reading

A more in-depth treatment of scenarios and architectural drivers can be found in L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed., Addison-Wesley, 2021. Also found in this book is an extensive discussion of architectural tactics, which are useful in guiding an architecture to achieve quality attribute goals. Likewise, this book contains an extensive discussion of QAW and Utility Trees.

A discussion of technical debt, including design/architecture debt, can be found in N. Ernst, J. Delange, and R. Kazman, *Technical Debt in Practice: How to Find It and Fix It*, MIT Press, 2021.

The Mission Thread Workshop is discussed in R. Kazman, M. Gagliardi, and W. Wood, “Scaling Up Software Architecture Analysis”, *Journal of Systems and Software*, 85,

1511–1519, 2012; and in M. Gagliardi, W. Wood, and T. Morrow, *Introduction to the Mission Thread Workshop*, Software Engineering Institute Technical Report CMU/SEI-2013-TR-003, 2013.

An overview of discovery prototyping, JRP, JAD, and accelerated systems analysis can be found in any competent book on systems analysis and design, such as J. Whitten and L. Bentley, *Systems Analysis and Design Methods*, 7th ed., McGraw-Hill, 2007.

An extensive collection of architectural design patterns for the construction of distributed systems can be found in F. Buschmann, K. Henney, and D. Schmidt, *Pattern-Oriented Software Architecture, Volume 4: A Pattern Language for Distributed Computing*, Wiley, 2007. Other books in the POSA (Patterns of Software Architecture) series provide additional pattern catalogs. Many other pattern catalogs specializing in particular application domains and technologies exist. A few examples are listed here:

- E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995.
- E. Fernandez-Buglioni, *Security Patterns in Practice: Designing Secure Architectures Using Software Patterns*, Wiley, 2013.
- J. Gilbert, *Software Architecture Patterns for Serverless Systems: Architecting for Innovation with Events, Autonomous Services, and Micro Frontends*, Packt Publishing, 2021.
- H. Percival and B. Gregory, *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices*, O'Reilly, 2020.

The “bible” for software architecture documentation is P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford, *Documenting Software Architectures: Views and Beyond*, 2nd ed., Addison-Wesley, 2011.

A chapter devoted to software architecture is now included in Version 4.0 of the SWEBOK: Hironori Washizaki, eds., *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, Version 4.0, IEEE Computer Society, 2024; www.swebok.org.

2.7 Discussion Questions

1. Can you find an example of an application that, at the time of its development, was considered to have a novel design? Why was its design novel?
2. What risks arise when an architect does all of the design of the system (including element interaction design and element internals design)? What happens if all of this design is performed initially, before any development begins?
3. Consider an application that you are familiar with. It might be one that you have worked on, or one you have used, such as a social network, a video streaming app, a productivity tool, or a game. What are drivers for this application?

various steps of the design process. These aspects include the use of a backlog, the various ways that ADD can be applied to different system contexts, the identification and selection of design concepts and their use in producing structures, the definition of interfaces, and the production of preliminary documentation.

Even though the overall architecture development life cycle includes documenting and analyzing architecture as activities that are separate from design, we have argued that a clean separation of these activities is artificial and harmful. We urge you to regularly perform at least some lightweight documentation and analysis activities and to consider them as integral parts of the design process.

4.10 Further Reading

Some of the concepts of ADD 3.0 were first introduced in H. Cervantes, P. Velasco, and R. Kazman, “A Principled Way of Using Frameworks in Architectural Design”, *IEEE Software*, 46–53, March/April 2013.

Version 2.0 of ADD was first documented in the SEI Technical Report: R. Wojcik, F. Bachmann, L. Bass, P. Clements, P. Merson, R. Nord, and B. Wood, *Attribute-Driven Design (ADD), Version 2.0*, SEI/CMU Technical Report CMU/SEI-2006-TR-023, 2006. An extended example of using ADD 2.0 was documented in W. Wood, *A Practical Example of Applying Attribute-Driven Design (ADD), Version 2.0*, SEI/CMU Technical Report: CMU/SEI-2007-TR-005.

The concept of an architecture backlog is discussed in C. Hofmeister, P. Kruchten, R. Nord, H. Obbink, A. Ran, and P. America, “A General Model of Software Architecture Design Derived from Five Industrial Approaches”, *Journal of Systems and Software*, 80, 106–126, 2007.

The ARID method is discussed in P. Clements, R. Kazman, and M. Klein, *Evaluating Software Architectures: Methods and Case Studies*, Addison-Wesley, 2002.

The CBAM method is presented in L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed., Addison-Wesley, 2013.

The ways in which an architecture can be documented are covered extensively in P. Clements et al. *Documenting Software Architectures: Views and Beyond*, 2nd ed., Addison-Wesley, 2011. More Agile approaches to documenting are discussed in books such as S. Brown, *Software Architecture for Developers*, Lean Publishing, 2015.

The importance and challenges of capturing design rationale are discussed in A. Tang, M. Ali Babar, I. Gorton, and J. Han, “A Survey of Architecture Design Rationale”, *Journal of Systems and Software*, 79(12), 1792–1804, 2007. A minimalistic technique for capturing rationale is discussed in U. Zdun, R. Capilla, H. Tran, and O. Zimmermann, “Sustainable Architectural Design Decisions”, *IEEE Software*, 30(6), 46–53, 2013.

4



The Architecture Design Process

In this chapter, we provide a detailed discussion of ADD, the design method that is at the heart of this book. We begin with an overview of the method and of each of its steps. This overview is followed by more detailed discussions of different aspects that need to be considered when performing these steps. We also discuss the identification and selection of design concepts, the production of structures from these design concepts, the definition of interfaces, the production of preliminary documentation, and, finally, a technique to track design progress.

Why Read This Chapter?

Architectural design is the foundation for any complex software system. Unfortunately, it is most often done in an ad hoc way, if at all. It is our belief, and our experience, that having a structured way of doing design results in better, more predictable outcomes. In this chapter, we introduce a method that allows design to be performed in a structured way; we hope to convince you that this approach truly leads to better outcomes.

4.1 The Need for a Principled Method

In Chapter 2, we discussed the various concepts associated with design. The question is, how do you actually perform design? Performing design to *ensure* that the drivers are satisfied requires a principled method. By “principled”, we refer to a method that takes into account all of the relevant aspects that are needed to produce an adequate design. Such a method provides guidance that is necessary to guarantee that your drivers are satisfied. To achieve this goal in a cost-effective, repeatable way, you need a method that guides you in combining and incorporating reusable design concepts.

Performing design adequately is important because architecture design decisions have significant consequences at different points in a project's lifetime. For example, during the estimation phase, an appropriate design will allow for a better estimation of cost, scope, and schedule. During development, an appropriate design will be helpful to avoid later rework and facilitate development and deployment. Finally, a clear understanding of what architectural design involves is necessary to better manage aspects of technical debt.

4.2 Attribute-Driven Design 3.0

Architecture design is performed in a series of *rounds* across the development of a software project. Each design round may take place within a project increment such as a sprint. Within these rounds, a number of *design iterations* are performed. Perhaps the most important characteristic of the ADD method is that it provides detailed, step-by-step guidance on the tasks that have to be performed during the design iterations.

ADD has been used successfully for more than 20 years. Version 2.0 was published in 2006, and version 3.0 was published in 2016, in the first edition of this book. While dramatic changes have occurred in the development landscape since the publication of version 3.0, the method itself remains unchanged. That attests to the general applicability of ADD.

We have added chapters in the book that explain how ADD can be used with the software development techniques that are widely used today, but the steps and artifacts of the method, which are shown in Figure 4.1 remain the same. This is good news for you as an architect!

In the following subsections, we provide an overview of the activities in each of the ADD steps. Note that we present them as linear steps, but there may be significant interactions among them, resulting in micro-iterations. This is the nature of design; it is not linear.

4.2.1 Step 1: Review Inputs

Before starting a design round, you need to make sure that the inputs to the design process are available and correct. First, you need to ensure that you are clear about the *purpose* for the design activities that will ensue. The purpose may be, for example, to produce a design for early estimation, to refine an existing design to build a new increment of the system, or to design and generate a prototype to mitigate certain technical risks (see Section 2.4.1 for a discussion of the design purpose). Also, you need to make sure that the other drivers needed for the design activity are available. These include primary functional requirements, quality attribute scenarios, architectural constraints, and concerns. Finally, if this is not the first design round, or if this is not greenfield development, an additional input that you need to consider is the existing architecture design.

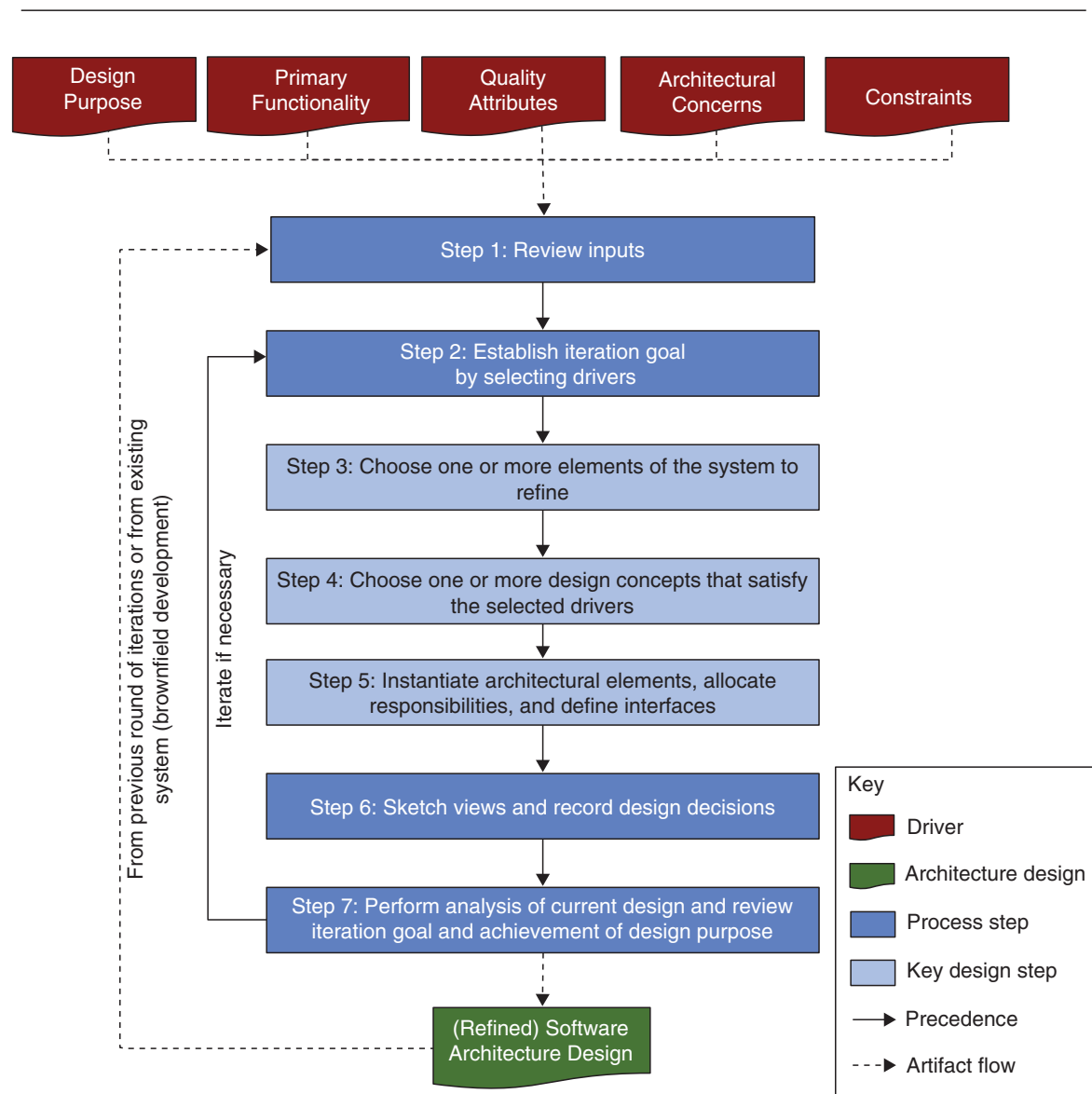


FIGURE 4.1 Steps and artifacts of ADD version 3.0

At this point, we assume that the primary functionality and quality attribute scenarios have been prioritized, ideally by your most important project stakeholders. (If not, you can employ various techniques to elicit and prioritize them, as discussed in Sections 2.4.2 and 2.4.3.) You, as the architect, must now “own” these drivers. You need to check, for example, whether any important stakeholders were overlooked in the original requirements elicitation process, or whether any business conditions have changed since the prioritization was

performed. These drivers really do “drive” design, so getting them right and getting their priority right is crucial. We cannot stress this point strongly enough. Software architecture design, like most activities in software engineering, is a “garbage in, garbage out” process. The results of ADD cannot be good if the inputs are poorly formed.

As a rule of thumb, you should be able to start designing if, besides the design purpose, constraints, and initial architectural concerns, you have established the primary use cases and the most important quality attribute scenarios. This, of course, does not mean you will make decisions focusing on only these drivers: You still need to address other quality attribute scenarios, use cases, and architectural concerns, but these can be treated later on.

The drivers become part of an architectural design backlog that you should use to perform the different design iterations. We discuss this idea in more depth in Section 4.8.1.

4.2.2 Step 2: Establish the Iteration Goal by Selecting Drivers

A design round comprises the architecture design activities performed within a development cycle if an iterative development model is used, or the whole set of architecture design activities if a waterfall model is used. Through one or more rounds, you produce an architecture that suits the established design purpose.

A design round generally takes the form of a series of design iterations, where each iteration focuses on achieving a particular goal. Such a goal typically involves designing to satisfy a subset of the drivers. For example, an iteration goal could be to create structures from elements that will support a particular performance scenario, or that will enable a use case to be achieved. For this reason, when performing design, you need to establish a goal before you start a particular design iteration.

Selecting an adequate goal for a design iteration can sometimes be challenging. In some cases, the goal may be too small (a unique and relatively simple driver is selected) or too big (too many drivers are selected). The following guidelines can help you establish a “right-sized” iteration goal:

- The goal may be to address a single important driver—for example, the architectural concern of decomposing the system or addressing a challenging quality attribute scenario.
- The goal may be to make decisions to satisfy a set of similar drivers—for example, making design decisions to address a family of use cases or related quality attribute scenarios.
- The goal may be to make decisions to satisfy a set of related drivers—for example, making design decisions to address a user story and an associated quality attribute scenario.

As we will discuss in Section 4.3, depending on the type of system whose architecture is being designed, there may be a “best”—or at least strongly suggested—ordering of the iteration goals that need to be addressed. For example, for a greenfield system in a mature domain, your initial goal is typically to identify an overall structure for the system by choosing a reference architecture.

4.2.3 Step 3: Choose One or More Elements of the System to Refine

This step is where the core design activities start. Satisfying drivers requires you to produce one or more architectural structures. These structures are composed of interrelated elements, and those elements are generally obtained by refining other elements that you previously identified in an earlier iteration. Refinement can mean decomposition into finer-grained elements (top-down approach), combination of elements into coarser-grained elements (bottom-up approach), or improvement of previously identified elements. For greenfield development, you can start by establishing the system context and then selecting the only available element—that is, the system itself—for refinement by decomposition. For existing systems or for later design iterations in greenfield systems, you would normally choose to refine elements that were identified in prior iterations.

The elements that you will select are the ones that are involved in the satisfaction of specific drivers. For this reason, when design is performed for an existing system, you need to have a good understanding of the elements that are part of the as-built architecture of the system. This may involve some “detective work”, reverse engineering, or discussions with developers.

We have presented steps 2 and 3 in the order they appear in the ADD method. That is to say, step 2 precedes step 3. However, in some cases you may need to reverse this order. For example, when you are designing a greenfield system or fleshing out certain types of reference architectures, you will—at least in the early stages of design—focus on elements of the system and start the iteration by selecting a particular element; only then will you consider the drivers that you want to address.

4.2.4 Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

Choosing the design concepts is probably the most difficult decision you will face in the design process, because it requires you to identify alternatives among design concepts that can be used to achieve your iteration goal, and to select one of these alternatives. As we saw in Section 3.2, different types of design concepts exist, and, for each type, there may be many options. This can result in a considerable number of alternatives that need to be analyzed to make a choice, though the existence of architecture standards may help to constrain the number of viable alternatives. In Section 4.4, we discuss the identification and selection of design concepts in more detail.

4.2.5 Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

Once you have selected one or more design concepts, you must make another design decision, which involves *instantiating* elements out of the design concepts that you selected. Instantiation means adjusting the selected design concepts to the problem at hand. For example, if you

selected the Layers pattern as a design concept, you must decide how many layers will be used, since the pattern itself does not prescribe a specific number. In this example, the layers are the elements that are instantiated. If you selected a particular availability tactic, such as “redundant spare”, instantiation implies deciding how this particular tactic will be achieved, such as by adding replication to a particular element. In certain cases, instantiation can mean configuration. For example, you may have dedicated an iteration to selecting technologies and associating them with the elements in your design. In further iterations, you might refine these elements by making finer-grained decisions about how they should be configured to support a particular driver, such as a quality attribute.

After instantiating the elements, you need to allocate responsibilities to each of them. For example, in a typical back-end of a web-based enterprise system, at least three layers are usually present: the API layer, the business layer, and the data layer. The responsibilities of these layers differ: The responsibilities of the API layer include exposing endpoints, whereas the responsibilities of the data layer include managing the persistence of data. An additional utility layer may be considered as well.

Instantiating elements is just one of the tasks you need to perform to create structures that satisfy a driver or a concern. The elements that have been instantiated also need to be connected, thereby enabling them to collaborate with one another. This requires the existence of *relationships* between the elements and the exchange of information through some kind of interface. The interface is a contractual specification of how information should flow between the elements. Section 4.5 provides more details on how the different types of design concepts are instantiated and how structures are created, and Section 4.7 discusses how interfaces can be defined.

The activities that are performed in this step are frequently performed by creating some type of diagram, either on a whiteboard or with some diagramming tool. Finally, it should be noted there may be a lot of back and forth (micro-iterations) among steps 3 and 5, as these are the central steps of the process in which design decisions are made.

4.2.6 Step 6: Sketch Views and Record Design Decisions

At this point, you have completed the design activities for the iteration. Nevertheless, you may not have taken any actions to ensure that the views—the representations of the structures you created—are preserved. For instance, if you performed the previous step in a conference room, you probably ended up with a series of diagrams on a whiteboard. This information is essential, and you need to capture it so that you can later analyze and communicate it to other stakeholders.

The views that you have created are almost certainly incomplete, so these diagrams may need to be revisited and refined in a subsequent iteration. This is typically done to accommodate elements resulting from other design decisions that you will make to support additional drivers. This factor explains why we speak of “sketching” the views in ADD—that is, creating a preliminary type of documentation. The more formal, more fully fleshed-out documentation of these views—should you choose to produce it—occurs only after a number of design iterations have been finished (as part of the architectural documentation activity discussed in Section 1.2.2).

In addition to storing the sketches of the views, you should record the significant decisions that are made in the design iteration, and the reasons that led to these decisions (i.e., the rationale), to facilitate later analysis and understanding of the decisions. For example, decisions about important tradeoffs might be recorded at this time. During a design iteration, decisions are primarily made in steps 4 and 5. Section 4.7 provides further information on how to create preliminary documentation *during* design, including creating sketches and recording design decisions and their rationales.

4.2.7 Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

By the time you reach step 7, you should have created a partial design that addresses the goal established for the iteration. Making sure that this is actually the case is a good idea, because it helps you avoid unhappy stakeholders and later rework. You can perform the analysis yourself by reviewing the sketches of the views and design decisions that you recorded, but an even better idea is to have someone else help you review this design. We recommend this practice for the same reason that organizations frequently establish a separate testing/quality assurance group: Another person will not share your assumptions, and will have a different experience base and a different perspective. Pulling in someone with a different point of view can help you find flaws, in both code and architecture. We discuss analysis in more depth in Chapter 11.

Once the design performed in the iteration has been analyzed, you should review the state of your architecture in terms of the established design purpose. This means considering whether you have performed enough design iterations to satisfy the drivers that are associated with the design round, as well as whether the design purpose has been achieved or if additional design rounds are needed in future project increments. Section 4.8 describes simple techniques that allow you to keep track of design progress.

4.2.8 Iterate If Necessary

Ideally, you should perform additional iterations and repeat steps 2 to 7 for every driver that was considered as part of the input. More often than not, such iterations are not possible because of time or resource constraints that force you to stop the design activities and move on to the next activities in the development process—typically implementation.

What are the criteria for evaluating whether more design iterations are necessary? We let *risk* be our guide. You should at least have addressed the drivers with the highest priorities. Ideally, you should have assured that the critical drivers are satisfied or, at least, that the design is “good enough” to satisfy them. Finally, when performing iterative development, you can choose to perform one design round in every project iteration. The initial rounds should focus on addressing the drivers with the highest priorities, while subsequent rounds focus on making design decisions for other drivers with lower priorities or on new drivers that appear as development progresses.

4.3 Applying ADD to Different System Contexts

When writing a paper or an essay, you might have experienced the much-dreaded “fear of the blank page”. Similarly, when you start designing an architecture, you may face a situation in which you ask yourself, “How do I begin designing?” To answer this question, you need to consider which type of system you are designing.

Design of software systems falls into three broad categories: (1) design of a *greenfield* system for a mature (i.e., well-known) domain; (2) design of a greenfield system for a domain that is novel (i.e., a domain that has a less established infrastructure and knowledge base); and (3) design for making changes to an existing system (*brownfield*). Each of these categories involves different uses of ADD and different ways that ADD can be applied to guide the design process.

4.3.1 Design of Greenfield Systems for Mature Domains

The design of a greenfield system for a mature domain occurs when you are designing an architecture for a system that is built from “scratch” and when this type of system is well known and understood—that is, when there is an established portfolio of tools and technologies, and an associated knowledge base. Examples of mature domains include the following:

- Traditional desktop applications
- Interactive applications that run on a mobile device
- Enterprise applications accessed from a web browser, which store information in a relational database, and which provide support for partially or fully automating business processes
- A microservice that manages a single domain entity

Because these types of applications are relatively common, some general architectural concerns associated with their design are well known, well supported, and well documented. If you are designing a new system that falls into this category, we recommend following the roadmap shown in Figure 4.2.

The goal of your *initial* design iteration(s) should be to address the general architectural concern of establishing an initial overall system structure. Is this to be a three-tier client-server application, a peer-to-peer application, a mobile app connecting to a Big Data back-end, or something else? Each of these options will lead you to different architectural solutions, and these solutions will help you to achieve your drivers. To achieve this iteration goal, you will select some design concepts. Specifically, you will typically choose one or more reference architectures and patterns, including patterns for deployment of the system (see Section 6.2). You may also select some externally developed components, such as frameworks. The types of frameworks that are typically chosen in early iterations are either “full-stack” frameworks that are associated with the selected reference architectures, or more specific frameworks that are associated with elements established by the reference architecture (see Section 3.2.1).

In this first iteration, you should review all of your drivers when selecting the design concepts, but you will probably pay more attention to the constraints and to quality attributes that are not associated with specific functionalities and that favor particular reference architectures or require particular deployment configurations. Consider an example: If you select a reference architecture for Big Data systems, you have presumably chosen a quality attribute such as low latency with high data volumes as your most important driver. Of course, you will make many subsequent decisions to flesh out this early choice, but this driver has already exerted a great influence on your design, such as the selection of a particular reference architecture.

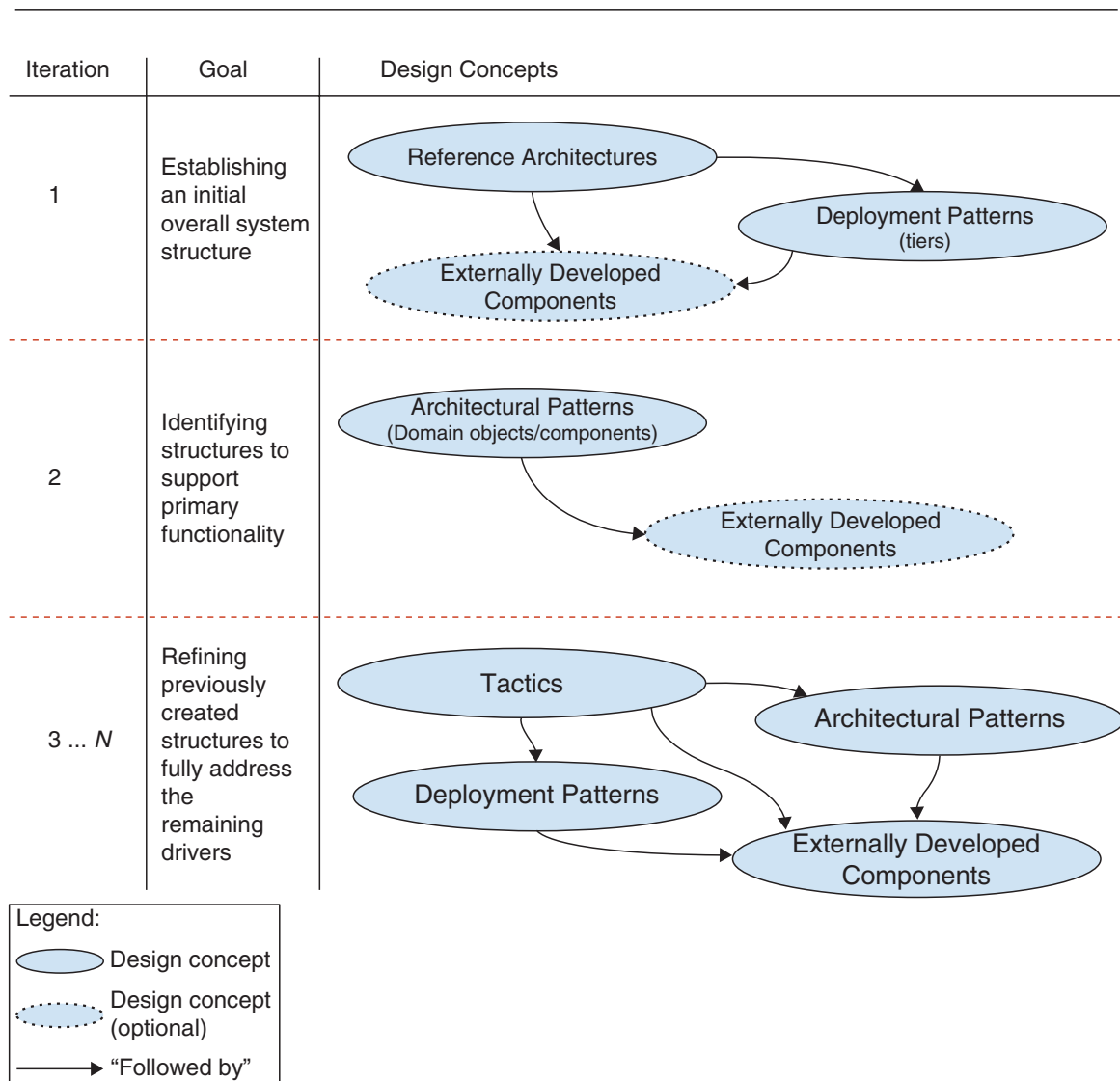


FIGURE 4.2 Design concept selection roadmap for greenfield systems

The goal of your *next* design iteration(s) should be to identify structures that support the primary functionality. As noted in Section 2.4.3, allocation of functionality (i.e., use cases or user stories) to elements is an important part of architectural design because it has critical downstream implications for modifiability and allocation of work to teams. Furthermore, once functionality has been allocated, the elements that support it can be refined in later iterations to support the quality attributes associated with these functionalities. For example, a performance scenario may be associated with a particular use case. Achieving the performance goal may require making design decisions across all of the elements that participate in the satisfaction of this use case. To allocate functionality, you usually refine the elements that are associated with the reference architecture by decomposing them. A particular use case may require the identification of multiple elements. For example, if you have selected a web application reference architecture, supporting a use case will probably require you to identify modules across the different layers associated with this reference architecture. Finally, at this point you should start thinking about allocating functionality—associated with modules—to (teams of) developers.

The goal of your *subsequent* design iterations should be to refine the structures you have created so far to fully address the remaining drivers. Addressing these drivers, and especially quality attributes, will likely require you to use the three major categories of design concepts—tactics, patterns, and externally developed components such as frameworks—as well as commonly accepted design best practices such as modularity, low coupling, and high cohesion. For example, to (partially) satisfy a performance requirement for the search use case in a web application, you might select the “maintain multiple copies of data” tactic and implement this tactic by configuring a cache in a framework that is used inside an element responsible for persisting data.

This roadmap is appropriate for the initial project iterations, but it is also extremely useful for early project estimation activities (see the discussion of designing to support estimation in Section 12.1.1). Why have we created such a roadmap? First, because the process of starting an architectural design is always complex. Second, because many of the steps in this roadmap are frequently overlooked or done in an intuitive and ad hoc way, rather than in a well-considered, reflective way. Third, because different types of design concepts exist, and it is not always clear at which point in the design they should be used. This roadmap encapsulates best practices that we have observed in the most competent architecture organizations. Simply put, the use of a roadmap results in better architectures, particularly for less mature architects.

4.3.2 Design of Greenfield Systems for Novel Domains

In the case of novel domains, it is more challenging to establish a precise roadmap, because reference architectures may not exist and there may be few, if any, externally developed components that you can use. You are, more than likely, working from first principles and creating your own home-grown solutions. Even in this case, however, general-purpose design concepts such as tactics and patterns can guide you, aided by strategic prototyping. In essence, your iteration goals will mostly be to continuously refine the previously created structures to fully address the drivers.

Many times, your design goal will focus on the creation of prototypes so that you can explore possible solutions to the challenge that you are facing. In particular, you may need to focus on quality attributes and design challenges oriented toward issues such as performance, scalability, or security. We discuss the creation of prototypes in Section 4.4.2.

Of course, the notion of “novel” is fluid. Mobile application development was a novel domain 15 or 20 years ago, but now it is a well-established field.

4.3.3 Design for an Existing System (Brownfield)

Architecture design for an existing system may occur for different purposes. The most obvious is maintenance—that is, when you need to satisfy new requirements or correct issues, and doing so requires changes to the architecture of an existing system. You may also be making architectural changes to an existing system for the purpose of *refactoring* (see Chapter 10). When refactoring, you change the architecture of an existing system, without altering its functions, to reduce technical debt, to introduce technology updates, or to fix quality attribute problems (e.g., the system is too slow, or insecure, or crashes frequently).

To choose which elements to decompose as part of the design process (step 3 of ADD), you need to first identify which elements are present in the architecture of the existing system. In this sense, before starting the design iterations, your first goal should be to ensure that you have a clear understanding of the existing architecture of the system.

Once you understand the elements, properties, and relationships that constitute the system’s architecture, and the characteristics of the existing code base, the design steps can be performed similarly to what is done for greenfield systems after the initial design iteration. Your design iteration goals here will be to identify and refine structures to satisfy architectural drivers, including new functionality and quality attributes, and to address specific architectural concerns. These design iterations will typically not involve establishing a new overall system structure unless you are dealing with a major refactoring.

4.3.4 Design to Replace a Legacy Application

As systems age, many of the technologies they were built upon become increasingly obsolete, such that their replacement can be both complex and expensive. Also, some of these systems have accumulated so much technical debt that its repayment becomes infeasible (see Chapter 10). When these situations occur, replacement of the system often emerges as the only feasible choice. But complex systems are difficult to replace all at once; instead, a gradual replacement approach is often less risky and more palatable to management.

Patterns such as the Strangler Fig offer some help in this task. This pattern allows an existing application to be replaced gradually while it keeps operating. The application to be replaced is typically moved behind a proxy, which initially delegates all of the service calls from clients to the legacy application. Parts of the legacy application then are redesigned and implemented—for example, following a microservices approach if the existing application is a monolith. Once the new parts are deployed, the proxy begins to route calls to them, without affecting existing clients. This approach proceeds gradually until all of the invocations that

were originally processed by the legacy application are now processed by newly developed components. At that point, the legacy application has been “strangled” and can be removed from service; the replacement is complete. The name “Strangler Fig” was chosen as an analogy to its botanical counterpart: This fig plant slowly surrounds an existing tree, which eventually dies once it is strangled.

When performing this kind of gradual replacement, design iteration goals focus on replacing one or more services provided by the legacy application. Each design iteration hones in on new architectural choices for those replacement services.

4.4 Identifying and Selecting Design Concepts

Freeman Dyson, the English physicist, once said, “A good scientist is a person with original ideas. A good engineer is a person who makes a design that works with as few original ideas as possible”. This remark is particularly relevant in the context of software architecture design: Most of the time you don’t need to, and shouldn’t, reinvent the wheel. Rather, your major design activities are to identify and select design concepts to address the challenges and drivers that you encounter across the design iterations. Design is still an original and creative endeavor, but the creativity resides in the appropriate identification of these existing solutions and then on combining and adapting them to the problem at hand.

4.4.1 Identification of Design Concepts

The identification of design concepts can appear to be daunting, because of the vast number of design concepts that exist. There are likely dozens of design patterns, tactics, and externally developed components that you could use to address any particular issue. Making things even more complicated, these design concepts are scattered across many different sources: in the popular press, in research literature, in books, and on the Internet. Moreover, in many cases, there is no canonical definition of a concept. Different sites, for example, will define the Broker pattern in different, largely informal ways. Finally, once you have identified the alternatives that might potentially help you achieve the design goals of the iteration, you need to select among them.

To identify which design concepts you need at a particular point, recall our earlier discussion of design for different system contexts. Different points in the design process usually require different types of design concepts. For example, when you are designing a greenfield system in a mature domain, the types of design concepts that will help you initially structure the system are reference architectures and deployment patterns. As you progress in the design process, you will use all of the categories of design concepts: tactics, architecture and design patterns, and externally developed components (see Chapter 3). Keep in mind that to address

a specific design problem, you can and often will use and combine different types of design concepts. For example, when addressing a security driver, you might employ a security pattern, a security tactic, a security framework, or some combination of these.

Once you have more clarity regarding the types of design concepts that you wish to use, you still need to identify alternatives—that is, design candidates. There are several ways to do so, although you will probably use a combination of these techniques rather than just a single method:

- *Leverage existing best practices.* You can identify alternatives for your required design concepts by making use of catalogs that are available in printed or online form. Some design concepts, such as patterns, are extensively documented; others, such as externally developed components, are documented in a less thorough way. The benefits of this approach are twofold: You can identify many alternatives, and you can leverage the considerable knowledge and experience of others. The downsides are that searching for and studying the information can require a considerable amount of time, the quality of the documented knowledge is often unknown, and the assumptions and biases of the authors are unknown.
- *Leverage your own knowledge and experience.* If the system you are designing is similar to other systems you have designed in the past, you will probably want to begin with some of the design concepts that you have used before. The benefit of this approach is that the identification of alternatives is performed rapidly and confidently. The downside is that you may end up using the same ideas repeatedly, even if they are not the most appropriate for all the design problems that you are facing, and even if they have been superseded by newer, better approaches. As the saying goes, “If you give a small child a hammer, all the world looks like a nail”.
- *Leverage the knowledge and experience of others.* As an architect, you have a wealth of background information and knowledge that you have gained through the years. This foundation varies from person to person, especially if the types of design problems they have addressed in the past differ. You can leverage this information by performing the identification and selection of design concepts with some of your peers through brainstorming.

4.4.2 Selection of Design Concepts

Once you have identified a list of alternative design concepts, you need to select which one is the most appropriate to solve the design problem at hand. You can achieve this in a relatively simple way—by creating a table that lists the pros and cons associated with each alternative, and selecting one of the alternatives based on those criteria and your drivers. Table 4.1 shows a simplified example of such a table used to support the selection of different deployment paradigms.

TABLE 4.1 Example of a Table to Support the Selection of Alternatives

Name of Alternative	Pros	Cons
Monolithic application	Easier to manage and deploy.	Updates require redeployment of the whole application. Horizontal scaling requires creating a replica of the entire application.
Microservices-based application	Services can be replicated and updated independently.	More complex management.

You may also need to perform a more in-depth analysis to select the best alternative. Methods such as CBAM (cost/benefit analysis method) or SWOT (strengths, weaknesses, opportunities, threats) can help you to perform this analysis (see the sidebar “The Cost/Benefit Analysis Method”).

The Cost/Benefit Analysis Method

CBAM is a method that guides the selection of design alternatives using a quantitative approach. This method is based on the idea that architectural strategies (i.e., combinations of design concepts) affect quality attribute responses, and that the level of each response in turn provides system stakeholders with some benefit called *utility*. Each architectural strategy provides a different level of utility, but also has a cost and takes time to implement. The idea behind CBAM is that by studying levels of utility and costs of implementation, particular architectural strategies can be selected based on their associated return on investment (ROI). CBAM was conceived to be performed after ATAM (architecture tradeoff analysis method), but it is possible to use CBAM during design—that is, prior to the moment where the architectural evaluation is performed.

CBAM takes as its input a collection of prioritized traditional quality attribute scenarios, which are then analyzed and refined with additional information. The addition is to consider several levels of response for each scenario:

- The worst-case scenario, which represents the minimum threshold at which a system must perform (utility = 0)
- The best-case scenario, which represents the level after which stakeholders foresee no further utility (utility = 100)
- The current scenario, which represents the level at which the system is already performing (the utility of the current scenario is estimated by stakeholders)
- The desired scenario, which represents the level of response that the stakeholders are hoping to achieve (the utility of the desired scenario is estimated by stakeholders)

Using these data points, we can draw a utility–response curve, like that shown in Figure 4.3. After the utility–response curve is mapped for each of the different scenarios, a number of design alternatives may be considered, and their expected response values can be estimated. For example, if we are concerned about mean time to failure, we might consider three different architectural strategies (i.e., redundancy options)—for example, no redundancy, cold spare, and hot spare. For each of these strategies, we

could estimate their expected responses (i.e., their expected mean times to failure). In the graph shown in Figure 4.3, point “e” represents one such option, which is placed on the curve based on its expected response measure.

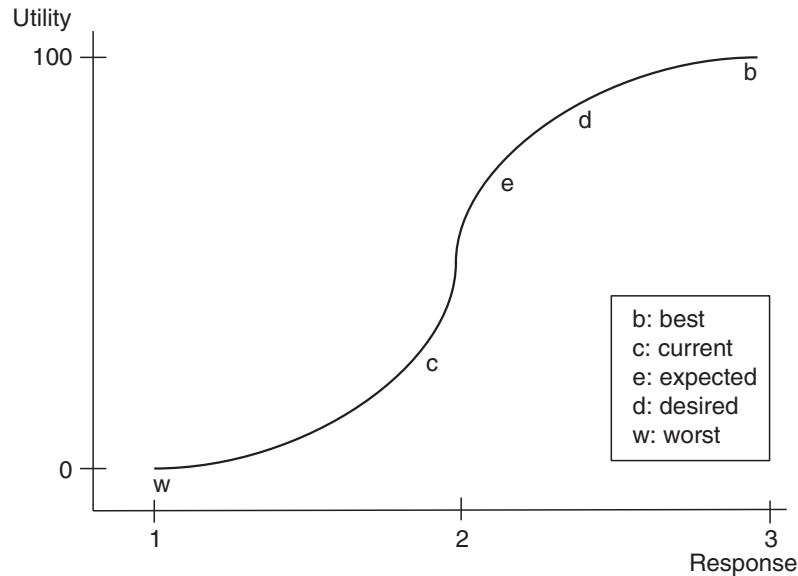


FIGURE 4.3 Utility–response curve

Using these response estimates, the utility values of each architectural strategy can now be determined via interpolation, which provides its expected benefit. The costs of each architectural strategy are also elicited: We would expect hot spare to be the most costly option, followed by cold spare and no redundancy.

Given all of this information, architectural strategies can then be selected based on their expected value for cost.

CBAM might seem relatively complex and time-consuming at first. However, some design decisions can have enormous economic consequences—in terms of their costs, their benefits, and their effects on project schedule. You must decide if you are willing to take the chance of making these decisions solely using a gut-feeling approach, or if it would be wiser to apply the more rational and systematic approach offered by CBAM.

If the aforementioned analysis techniques do not guide you to make an appropriate selection, you may need to create throwaway prototypes and collect measurements from them. The creation of early throwaway prototypes is a useful technique that can facilitate the selection of externally developed components. This type of prototype is usually created in a “quick and dirty” fashion without too much consideration of its maintainability or reuse. For these reasons, throwaway prototypes should not be used as a basis for further development.

Although the creation of prototypes can be costly compared to analysis (the ratio of costs is between 10 and 5 to 1, according to our sources), certain scenarios strongly motivate the creation of prototypes. Aspects that you should consider when deciding whether you will create a prototype include the following:

- Does the project incorporate emerging technologies?
- Is the technology new to the company?
- Are there certain drivers, particularly quality attributes, whose satisfaction using the selected technology presents risks (i.e., it is not understood whether they can be satisfied)?
- Is there a lack of trusted information, internal or external, that provides some degree of certainty that the selected technology will be useful to satisfy the project drivers?
- Are there configuration options associated with the technology that need to be tested or understood?
- Is it unclear whether the selected technology can be integrated with other technologies that are used in the project?

If most of your answers to these questions are “yes”, then you should strongly consider creating a throwaway prototype.

When identifying and selecting design concepts, you need to keep in mind the *constraints* that are part of the architectural drivers, because some constraints will restrict you from selecting particular alternatives. For example, a constraint might require that all libraries and frameworks in the system do not use the GPL license; thus, even if you have found a framework that could be useful for your needs, you may need to discard it if it has a GPL license. Also, the design concepts you selected in previous iterations may restrict the design concepts that you can select in the future because of incompatibilities. For example, if you selected a web application reference architecture for use in an initial iteration, you cannot select a user interface framework intended for local applications in a subsequent iteration.

Finally, you need to recognize that even though ADD provides guidance on how to perform the design process, it cannot ensure that you will make appropriate design decisions. Thorough reasoning and considering different alternatives (not just the first thing that comes to mind) are the best means to improve your odds of finding a good solution. We discuss doing “analysis in the design process” in Chapter 11.

4.5 Producing Structures

Design concepts per se won’t help you satisfy your drivers unless you produce *structures*; that is, you need to identify and connect elements that are derived from the selected design concepts. This process constitutes the *instantiation* of architectural elements in ADD: creating elements and relationships between them, and associating responsibilities with these elements.

It is important to remember that the architecture of a software system is composed of a set of structures, which can be grouped into three major categories:

- *Module structures*: composed of logical and static elements that exist at development time, such as files, modules, and classes
- *Component and connector (C&C) structures*: composed of dynamic elements that exist at runtime, such as processes and threads
- *Allocation structures*: composed of both software elements (from a module or a C&C structure) and non-software elements that may exist both at development time and at runtime, such as file systems, infrastructure, and development teams

When you instantiate a design concept, you may actually produce more than one structure. For example, in a particular iteration you might instantiate the Layers pattern, which results in a module structure. As part of instantiating this pattern, you will need to choose the number of layers, their relationships, and the specific responsibilities of each layer. As part of the iteration, you might also study how a scenario is supported by the elements that you have just identified. For example, you could create instances of the logical elements in a C&C structure and model how they exchange messages (see Section 4.6.2). Finally, you might decide who will be responsible for implementing the modules inside each of the layers, which is an allocation decision.

4.5.1 Instantiating Elements

The instantiation of architectural elements depends on the type of design concept that you are working with:

- *Reference architectures*. In the case of reference architectures, instantiation typically means that you perform some sort of customization. As part of this work, you will add or remove elements that are part of the structure that is defined by the reference architecture. For example, if you are designing a microservice, you will probably need a service reference architecture that includes service, business, and data layers, as we showed in Chapter 3.
- *Architectural and design patterns*. These patterns provide a generic structure composed of elements, their relationships, and their responsibilities. As this structure is generic, you will need to adapt it to your specific problem. Instantiation usually involves transforming the generic structure defined by the pattern into a specific one that is adapted to the needs of the problem that you are solving. For example, consider the Pipe and Filters architectural pattern. It establishes the basic elements of computation (filters) and their relationships (pipes), but does not specify how many filters you should use for your problem or what their relationships should be. You instantiate this pattern by defining how many pipes and filters are needed to solve your problem, by establishing the specific responsibilities of each of the filters, and by defining their topology.
- *Deployment patterns*. Similar to the case with architectural and design patterns, the instantiation of deployment patterns generally involves the identification and specification of infrastructure elements. For example, if you are using a Load-Balanced Cluster

pattern, instantiation may involve identifying the number of replicas to be included in the cluster, the load-balancing algorithm, and the physical location of the replicas. When deploying to the cloud, instantiation of these patterns involves selecting resources offered by the chosen cloud provider.

- *Tactics.* This design concept does not prescribe a particular structure, so you will need to use other design concepts to instantiate a tactic. For example, you might select a security tactic of authenticating actors and instantiate it by creating a custom-coded ad hoc solution, or by using a security pattern, or by using an externally developed component such as a security framework.
- *Externally developed components.* The instantiation of these components may or may not imply the creation of new elements. For example, in the case of object-oriented frameworks, instantiation may require you to create specific classes that inherit from the base classes defined in the framework. This will result in new elements. Other approaches, which do not involve the creation of new elements, might include choosing a specific technology from a technology family that was identified in a previous iteration, associating a particular framework to elements that were identified in a previous iteration, or specifying configuration options for an element associated with a particular technology (such as a number of threads in a thread pool).

4.5.2 Associating Responsibilities and Identifying Properties

When you are creating elements by instantiating design concepts, you need to consider the responsibilities that are allocated to these elements. For example, if you instantiate the Layers pattern and decide to use the traditional three-layer structure, you might decide that one of the layers will be responsible for managing the interactions with the users (typically known as the presentation layer). When instantiating elements and allocating responsibilities, you should keep in mind the high cohesion/low coupling design principle: Elements should have high cohesion (internally), defined by a narrow set of responsibilities, and low coupling (externally), defined by a lack of knowledge of the implementation details of other elements.

One additional aspect that you need to consider when instantiating design concepts is the properties of the elements. This may involve aspects such as the configuration options, statefulness, resource management, priority, or even hardware characteristics (if the elements that you created are physical nodes) of the chosen technologies. Identifying these properties supports analysis and the documentation of the design rationale.

4.5.3 Establishing Relationships Between the Elements

The creation of structures also requires making decisions with respect to the relationships that exist between the elements and their properties. Once again, consider the Layers pattern. You may decide that two layers are connected, but these layers will eventually be allocated to components that are, in turn, allocated to hardware. In such a case, you need to decide how communication will take place between these layers, as they have been allocated to components: Is the communication synchronous or asynchronous? Does it involve some type of network

communication? Which type of protocol is used? How much information is transferred and at what rate? These design decisions can have a significant impact with respect to achieving certain quality attributes such as performance.

4.6 Defining Interfaces

Interfaces are the externally visible properties of elements that establish a contractual specification that allows elements to collaborate and exchange information. There are two categories of interfaces: external and internal.

4.6.1 External Interfaces

External interfaces include application programming interfaces (APIs) from other systems that are *required* by the system that you are developing and interfaces that are *provided* by your system to other systems. Required APIs are part of the constraints for your system, as you usually cannot influence their specification. Chapter 5 provides an extensive discussion of APIs, including mechanisms for defining the APIs that are necessary when your system is responsible for providing them.

Establishing a system context at the beginning of the design process is useful to identify external interfaces. This context can be represented using a system context diagram, as shown in Figure 4.4. Given that external entities and the system under development interact via interfaces, there should be at least one external interface per external system (each relationship in the figure).

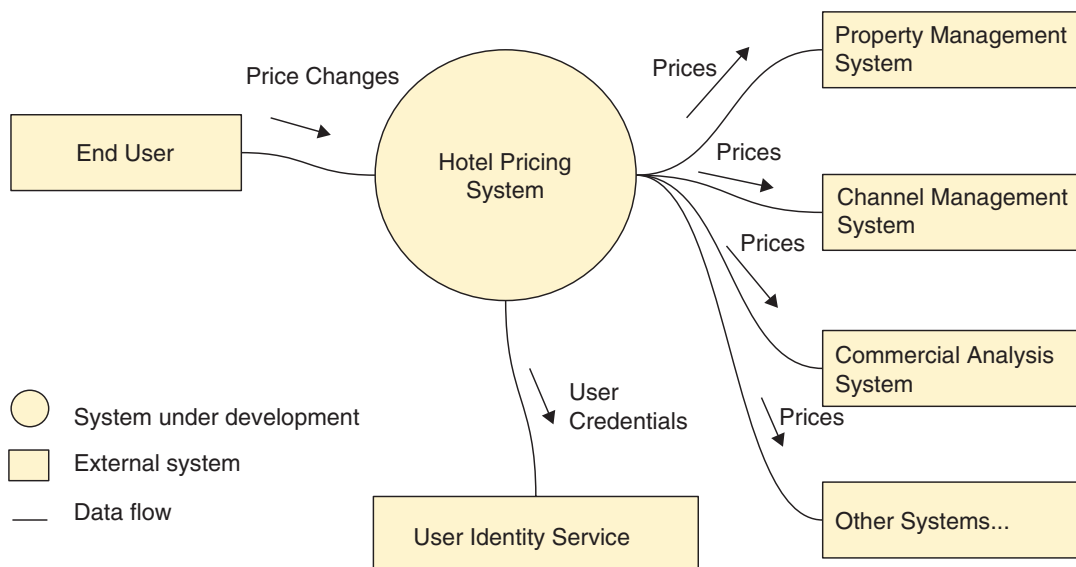


FIGURE 4.4 A system context diagram

4.6.2 Internal Interfaces

Internal interfaces are interfaces between the elements that result from the instantiation of design concepts. To identify the relationships and the interface details, you generally need to understand how the elements exchange information at runtime. You can achieve this with the help of modeling tools such as UML sequence diagrams (Figure 4.5), which allow you to model the information that is exchanged between elements during execution to support use cases or quality attribute scenarios. This type of analysis is also useful for identifying relationships between elements: If two elements need to exchange information directly, then a relationship between these elements must exist. The information that is exchanged becomes part of the specification of the interface.

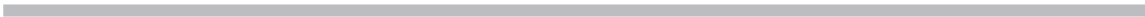


Figure 4.5 is an initial sequence diagram for HPS-2 (Change Prices) on the command side. The diagram shows how the client application invokes the POST method that triggers a change in prices. The PriceService retrieves the hotel and price that must be changed and asks the hotel object to calculate additional prices. To increase performance, an index must be added to the database (increase efficiency of resource usage tactic). Once prices are calculated, an event is created and sent to the PriceChangeEventProcessor, which in turn sends it to Kafka.

Key: UML
From this interaction, initial methods for the interfaces of the interacting elements can be identified:

PriceServices:

Method name	Description
updatePriceForPeriod	Update prices for a hotel and produce a price change event. Parameters: - Hotel identifier - Range of dates of price changes - Amount Returns: - True if the update is performed successfully. Throws: - PriceChangeEventException in case price change fails.

Note: More details of this example are presented in Chapter 8.

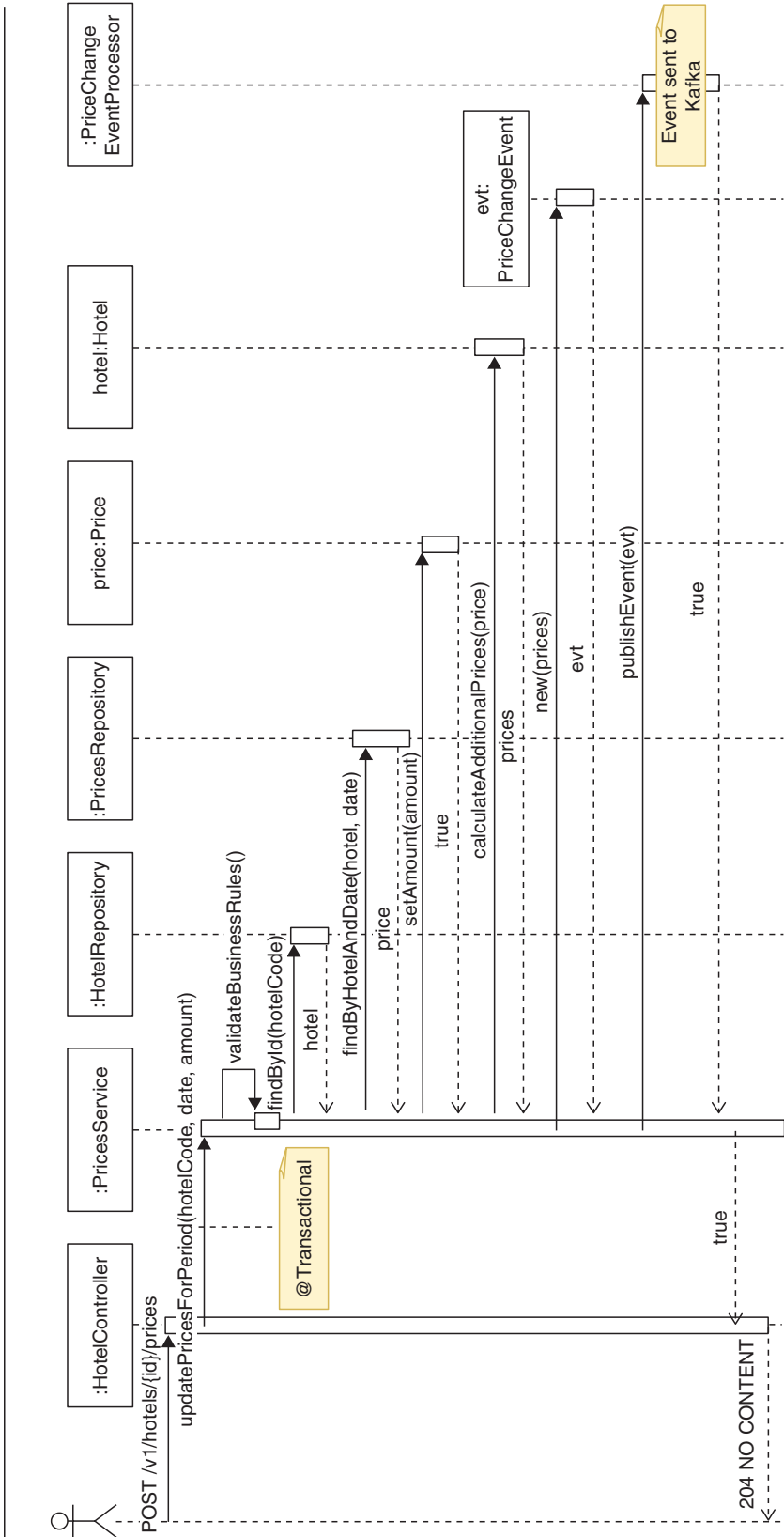


FIGURE 4.5 A sequence diagram used to identify interfaces

Interfaces typically consist of a set of operations (such as methods) with specified parameters, return values, and, possibly, exceptions and pre and post conditions. Some interfaces, however, may involve other information exchange mechanisms, such as a component that writes information to a file or database and another component that then accesses this information. Interfaces may also establish quality of service agreements. For example, the execution of an operation specified in the interface may be time constrained to satisfy a performance quality attribute scenario.

Identification of interfaces is usually not carried out to the same extent across all design iterations. When you are starting the design of a greenfield system, for example, your first iterations will produce only abstract elements such as layers, with these elements then being refined in later iterations. The interfaces of abstract elements such as layers are typically underspecified. For example, in an early iteration you might simply specify that the UI layer sends “commands” to the business logic layer, with the business logic layer then sending “results” back. As the design process advances, and particularly when you create structures to address specific use cases and quality attribute scenarios, you will need to refine the interfaces of the specific elements that participate in this interaction.

In some special cases, identification of interfaces is greatly simplified. For example, when you use a persistence framework that provides modules in the data layer, interfaces are already defined by the technology. The specification of interfaces then becomes a relatively trivial task, as the chosen technologies are designed to interoperate and hence have already “baked in” many interface assumptions and decisions.

Finally, you need to consider that not all of the internal interfaces of the system element will be identified as part of the design process (see the sidebar “Identifying Interfaces in Element Interaction Design”).

Identifying Interfaces in Element Interaction Design

Although defining interfaces is an essential part of the architecture design process, it is important to recognize that not all of the internal interfaces are identified during architectural design. As part of the architecture design process, you typically consider the primary use cases as part of the architectural drivers, and you identify elements (usually modules) that support this primary functionality along with the other drivers. This process will, however, not uncover *all* of the elements and interfaces for the system that are required to support the entire set of use cases. This lack of specificity is intended: Architecture is about abstraction, so some information is necessarily less important, particularly in the earliest stages of design.

Identifying the modules that support the nonprimary use cases is often necessary for estimation or work-assignment purposes. Identifying their interfaces is also necessary to support the individual development and integration of the modules and to perform unit testing. This identification of modules may be done early in the project life cycle, but it must not be confused with a big design up front (BDUF) approach. But in certain contexts, such as early estimation or iteration planning, this kind of work is hard to avoid.

As an architect, you may identify the set of modules that support the complete set of use cases for the system or for a particular release of the system. However, the identification of the interfaces associated with the modules that support the nonprimary use cases is typically not your responsibility, as it would require a significant amount of your

time and rarely has a major architectural impact. This task, which we call element interaction design (see Section 2.2.2), is usually performed after architectural design ends but before the development of (most of) the modules begins. Although this task should be performed by other members in the development team, as an architect you will play a critical role in shaping it, since these interfaces must adhere to the architectural design that you established. You, as the architect, must communicate the architecture to the engineers who are responsible for identifying the interfaces and ensure that they understand the rationale for the existing design decisions.

A good way to achieve this communication is to use the *active reviews for intermediate design (ARID)* method. In this method, the architecture design (or part of it) is presented to a group of reviewers—in this case, the engineers who will make use of this design. After the design presentation, the participants in the review select a set of scenarios. These scenarios are used for the core of the exercise, where the reviewers use the elements present in the architecture to satisfy them. In standard ARID, the reviewers are asked to write code or pseudo-code for the purpose of identifying interfaces. Alternatively, the architect can present the architecture, select a nonprimary functional scenario, and ask the participants to identify the interfaces of the components that support the scenario using sequence diagrams or a similar method.

Aside from the fact that the architectural design is reviewed in this exercise, the ARID approach offers additional benefits. Specifically, in a single meeting, the architecture design (or part of it) is presented to the entire team, and agreements can be reached with respect to how the interfaces should be defined (e.g., the level of detail or aspects such as parameter passing, data types, or exception management).

4.7 Creating Preliminary Documentation During Design

A software architecture is typically documented as a set of *views*, which represent the different structures that compose the architecture. The formal documentation of these views is not part of the design process. Structures, however, are produced as part of design. Capturing them, even in an informal manner (i.e., as sketches), along with the design decisions that led you to create these structures, is a task that should be performed as part of normal design activities.

4.7.1 Recording Sketches of the Views

When you produce structures by instantiating the design concepts that you have selected to address a particular design problem, you typically will not produce these structures in your mind, but rather will create some *sketches* of them. In the simplest case, you will produce these sketches on a whiteboard, a flip-chart, or even a piece of paper. Alternatively, you might use a modeling tool in which you will draw the structures. The sketches that you produce are the initial documentation for your architecture that you should capture and can flesh out later, if necessary. When you create sketches, you don't need to always use a more formal language such as UML. If you use some informal notation, you should at least be careful in maintaining consistency in the use of symbols. Eventually, you will need to add a legend to your diagrams to provide clarity and avoid ambiguity.

You should develop discipline in writing down the responsibilities that you allocate to the elements as you create the structures. The reasons for this are simple: As you identify an element, you are determining some responsibilities for that element in your mind. Writing it down *at that moment* ensures that you won't have to remember it later. Also, it is easier to write down the responsibilities associated with your elements gradually, instead of compiling all of them at a later time.

Creating this preliminary documentation as you design requires some discipline. But the benefits are worth the effort—you will be able to produce the more detailed architecture documentation relatively easily and quickly at a later point. One simple way that you can document responsibilities if you are using a whiteboard, a flip-chart, or a PowerPoint slide is to take a photo of the sketch that you have produced and paste it in a document, along with a table that summarizes the responsibilities of every element depicted in the diagram (Figure 4.6 provides an example). If you are using a computer-aided software engineering (CASE) tool, you can select an element to create and use the text area that usually appears in the properties sheet of the element to document its responsibilities, and then generate the documentation automatically.

Figure 4.6 is a sketch of a module view depicting the overall system structure for a CQRS-based system. The diagram is complemented with a table that describes the responsibilities of the various elements.

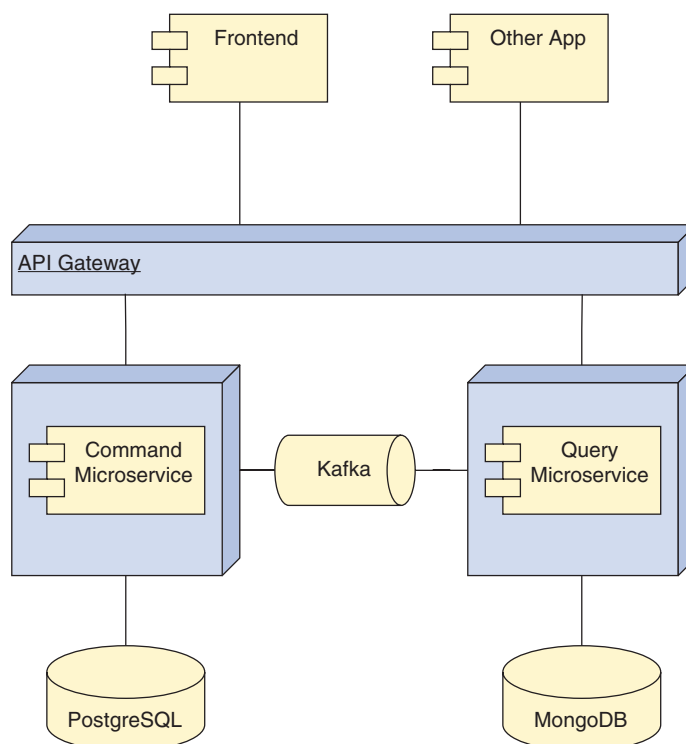


FIGURE 4.6 Sample preliminary documentation

Key: UML

Element	Responsibility
Command microservice	This microservice is responsible for making changes to the domain entities and sending an event when changes are made.
Query microservice	This microservice receives events sent by the command microservice and persists the information in the database to support querying.
...	...

Of course, it is not necessary to document *everything*. The three purposes of documentation are analysis, construction, and education. At the moment you are designing, you should choose a documentation purpose and then document to fulfill that purpose, based on your risk mitigation concerns. For example, if you have a critical quality attribute scenario that your architecture design needs to satisfy, and if you will need to prove this requirement is met in an analysis, then you must take care to document the information that is relevant for the analysis to be satisfactory. Alternatively, if you anticipate having to train new team members, then you should make a sketch of a C&C view of the system, showing how it operates and how the elements interact at runtime, and perhaps construct a crude module view of the system, showing at least the major layers or subsystems.

Finally, it is a good idea to remember, as you are documenting, that your design may eventually be analyzed. Consequently, you need to think about which information should be documented to support this analysis (see the sidebar “Scenario-Based Documentation”).

Scenario-Based Documentation

The analysis of an architecture design is based on your most important use cases and quality attribute scenarios. Simply put, a scenario is selected and you must explain to reviewers how the architecture supports the scenario, and justify your decisions. To start preparing for the analysis *while* you design, it is useful to produce and document structures that contain the elements that are involved in the satisfaction of a scenario. These artifacts should come naturally given that the design process is guided by scenarios, but keeping this point firmly in mind is always helpful.

During the design process, you should at least try to capture the following elements in a single document:

- The primary presentation: the diagram that represents the structure that you produced
- The element responsibilities table: to help you record the responsibilities of the elements that are present in the structure
- The relevant design decisions, and their rationales (see Section 4.7.2)

You might also capture two other pieces of information:

- A runtime representation of the element’s interaction—for example, a sequence diagram
- The initial interface specifications (which can also be captured in a separate document)

As you can see, all of this information needs to be produced as part of the design process. One way or another, you need to decide which elements are present in the system and how they interact. The only question is whether you bother to write this information down, or whether its sole representation is in the code.

If you follow the approach that we advocate here, at the end of the design you will have a set of preliminary views documented, with each of the views being associated with a particular scenario, and you will have constructed this documentation at little cost. This preliminary documentation can be used “as is” to analyze the design, and particularly through scenario-based evaluations.

4.7.2 Recording Design Decisions

In each design iteration, you make important design decisions to achieve your iteration goal. As we saw previously, these design decisions include the following:

- Selecting a design concept from several alternatives
- Creating structures by instantiating the selected design concept
- Establishing relationships between elements and defining interfaces
- Allocating resources (e.g., people, hardware, computation)
- Others

When you study a diagram that represents an architecture, you are seeing the end product of a thought process, but it may not be easy to understand the decisions that were made to achieve this result. Recording design decisions *beyond* the representation of the chosen elements, relationships, and properties is fundamental to help clarify how you arrived at the result: the design rationale.

When your iteration goal involves satisfying a specific quality attribute scenario, some of the decisions that you make will significantly influence your ability to achieve the scenario response measure. These are, therefore, the decisions that you should take the greatest care in recording. You should record these decisions because they are essential to facilitate analysis of the design you created; then to facilitate implementation; and, still later, to aid in understanding the architecture (e.g., during maintenance). Also *every* design decision is “good enough” but seldom optimal, so you need to justify the decisions made, and possibly revisit the remaining risks later.

You might think that recording design decisions is a tedious task. In reality, depending on the criticality of the system being developed, you can adjust the amount of information that is recorded. For example, to record a minimum of information, you can use a simple table such as that shown in Table 4.2. If you decide to record more than this minimum, the following information can prove useful:

- What evidence was produced to justify decisions?
- Who did what?
- Why were shortcuts taken?
- Why were tradeoffs made?
- What assumptions did you make?

TABLE 4.2 Example of a Table to Document Design Decisions

Design Decisions and Location	Rationale and Assumptions
Use a relational database for the Command microservice	Given that the Command microservice manages the different entities of the domain model (Hotels, Rates, Room Types, etc.), and given that these entities are related to each other, the most natural and appropriate mechanism for storing them is a relational database. Discarded alternatives are nonrelational databases or other types of storage.
Use a nonrelational database for the Query microservice	Because the database on the query side does not need to handle transactions and relationships between entities, and because the prices that are stored have a variable structure depending on the hotel, it was decided that the use of a nonrelational database is more appropriate. PriceChangeEvents can be appropriately stored in a document database. Discarded alternatives include other types of NoSQL, relational databases or other types of storage.

Much in the same way that you record responsibilities as you identify elements, you should record the design decisions as you make them. The reason for this is simple: If you leave it until later, you may not remember why you did things.

4.8 Tracking Design Progress

Although ADD provides clear guidelines to help you perform design systematically, it does not provide a mechanism to help you track design progress. When you are performing design, however, you will want to answer several questions:

- How much design do we need to do?
- How much design has been done so far?
- Are we finished?

Agile practices such as the use of backlogs and Kanban boards can help you track the design progress and answer these questions. These techniques are not limited to Agile methods, of course. Any development project using any methodology should track progress.

4.8.1 Use of an Architectural Backlog

The concept of an architecture (or design) backlog has been proposed by several authors. This is similar to the product backlog that is found in Agile development methods such as Scrum, but the focus here is not on feature development. Instead, the basic idea is that you need to create a list of the pending actions that still need to be performed as part of the architecture design process. These actions are called “enablers” in methods such as the Scaled Agile Framework.

Initially, you should populate the design backlog with tasks focused on directly satisfying drivers. Then, other activities that support the design of the architecture can also be included. For example:

- Creation of a prototype to test a particular technology or to address a specific quality attribute risk
- Exploration and understanding of existing assets (possibly requiring reverse engineering)
- Issues uncovered in a review of the design
- Review of a partial design that was performed on a previous iteration

For example, when using Scrum, the sprint backlog and the design backlog are not independent: Some features in the sprint backlog may require architecture design to be performed, so they will generate items that go into the architectural design backlog. While these two backlogs can be managed separately, it may be more practical to combine the architectural tasks with features and other elements in the backlog (see Figure 10.1, “Value and visibility: the dimensions of entries in your project backlog”).

Also, additional architectural concerns may arise as decisions are made. For example, if you choose a reference architecture, you will probably need to add specific architectural concerns, or quality attribute scenarios derived from them, to the architectural design backlog. An example of such a concern is the establishment of a mechanism to support observability.

4.8.2 Use of a Design Kanban Board

As design is performed in rounds and as a series of iterations within these rounds, you need to have a way of tracking the design’s progress. You must also decide whether you need to continue making more design decisions (i.e., performing additional iterations). One tool that can be used to facilitate this task is a Kanban board, like the one shown in Figure 4.7.

At the beginning of a design round, the inputs to the design process become entries in the backlog. Initially, that activity occurs in step 1 of ADD; the different entries in your backlog for this design round should be added to the “Not Yet Addressed” column of the board (unless you have some entries that were not concluded in previous design rounds that you wish to address here). When you begin a design iteration, in step 2 of ADD, the backlog entries that correspond to the drivers that you plan to address as part of the design iteration goal should be moved to the “Partially Addressed” column. Finally, once you finish an iteration and the analysis of your design decisions reveals that a particular driver has been addressed (step 7 of ADD), the entry should be moved to the “Completely Addressed” column of the board. It is important to establish clear criteria that will allow a driver to be moved to the “Completely Addressed” column (think of this as the “Definition of Addressed” criteria, similar to the “Definition of Done” criteria used in Scrum). A criterion may be, for example, that the driver has been analyzed or that it has been implemented in a prototype. Also, drivers that are selected for a particular iteration may not be completely addressed in that particular iteration, in which case they should remain in the “Partially Addressed” column. In preparation for subsequent iterations, you should consider how they can be allocated to the elements that exist at this point.

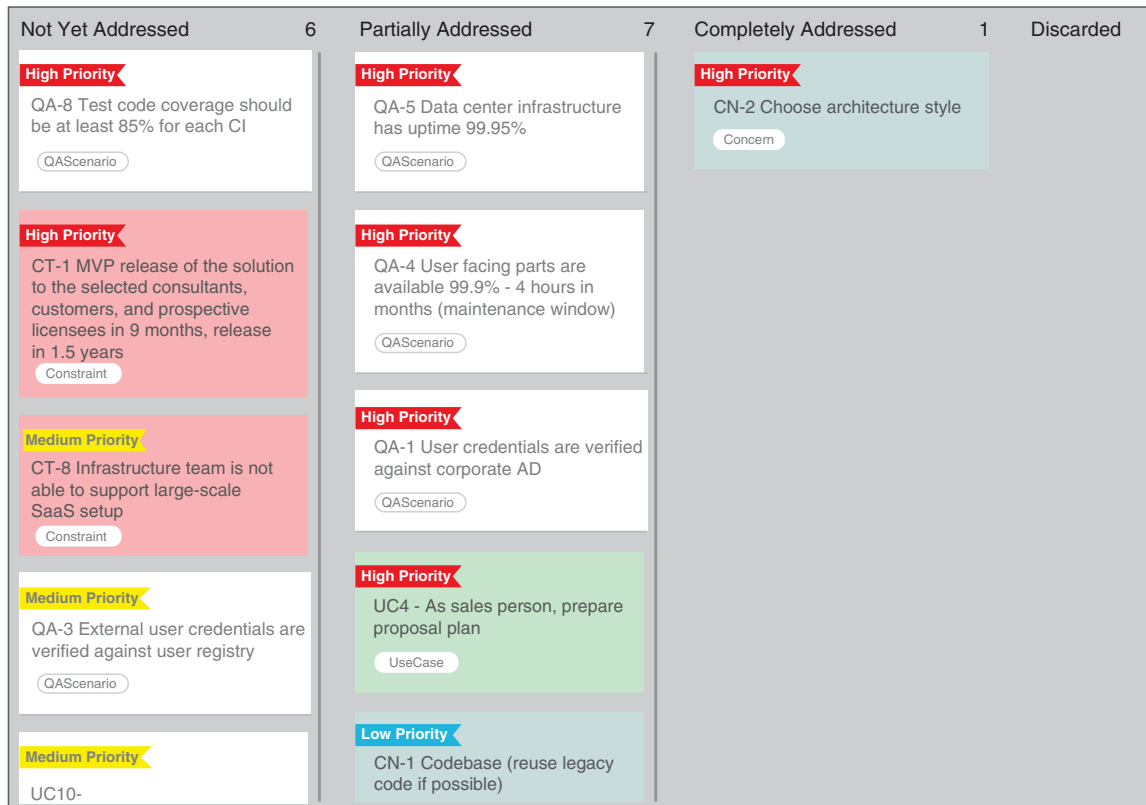


FIGURE 4.7 An example of a Kanban board used to track design progress

It can be useful to select a technique that will allow you to differentiate the entries in the Kanban board according to their priority. For example, you might use different colors of Post-it notes depending on the priority.

With a Kanban board, it is easy to visually track the advancement of design, as you can quickly see how many of the (most important) drivers are being or have been addressed in the design round. This technique also helps you decide whether you need to perform additional iterations as, ideally, the design round is terminated when a majority of your drivers (or at least the ones with the highest priority) are located under the “Completely Addressed” column.

4.9 Summary

In this chapter, we presented a detailed walk-through of the Attribute-Driven Design method, version 3.0. We also discussed several important aspects that need to be considered in the